

Geometric Differences in Activation Regions of MLPs under Different Loss Functions

Bachelor's thesis
in partial fulfillment of the requirements for the degree of
Bachelor of Science

University of Münster
Department of Computer Science

Supervisor:
Christof Duhme

First assessor:
Prof. Dr. Xiaoyi Jiang

Second assessor:
Prof. Dr. Malte Schilling

Submitted by:
Michael Kittel

Münster, July 2026

Geometric Differences in Activation Regions of MLPs under Different Loss Functions

---

Geometrische Unterschiede in den Aktivierungsregionen von MLPs unter verschiedenen Verlustfunktionen

Abstract

This bachelor's thesis investigates whether the choice of loss function influences activation-region geometry in rectified linear unit (ReLU) multilayer perceptrons (MLPs) and compares its effect with those of architecture depth and dataset difficulty. We compare models trained with harmonic loss (HL), cross-entropy (CE), and mean squared error (MSE) using three fully connected ReLU architectures that consist of one, two or three hidden layers. The models are evaluated on the two-dimensional binary classification datasets: `moons`, `circles`, and `spiral`. In addition to predictive accuracy, the analysis uses activation-region counts and area distributions, interactions between activation regions and the decision boundary, and distances from data points to activation and decision boundaries. Activation regions are approximated using a grid-based method whose reliability is evaluated against feasible-region counts. All three loss functions achieve similar predictive performance within most experimental settings, while the `spiral` dataset remains substantially more difficult. We observe only small and setting-specific geometric differences between the loss functions. HL produces higher decision-boundary fractions and larger boundary-region areas in some settings. On the other hand, architecture depth and dataset geometry have stronger effects on the activation-region partition. In this study, deeper networks form more activation regions, have smaller regions on average, and show more unequal region-area distributions. On the more difficult `spiral` dataset, a larger proportion of the activation-region structure is located near the decision boundary. During the training phase, activation-region geometry stabilizes in almost all settings, but frequently later than predictive accuracy. These results indicate that the activation regions reorganize after classification performance reaches stability. Overall, architecture depth and dataset difficulty are therefore the dominant factors in our experiment that influence activation-region geometry, while the effect of the loss function is comparatively small.

Diese Bachelorarbeit untersucht, ob die Wahl der Verlustfunktion die Geometrie der Aktivierungsregionen in mehrschichtigen Perzeptrons (MLPs) mit Rectified-Linear-Unit-Aktivierungen (ReLU) beeinflusst. Dabei wird ihr Einfluss mit den Auswirkungen der Architekturtiefe und der Schwierigkeit des Datensatzes verglichen. Wir vergleichen Modelle, die mit harmonic loss (HL), cross-entropy (CE) und mean squared error (MSE) trainiert wurden. Dafür verwenden wir drei ReLU-Architekturen, mit einer, zwei und drei verborgenen Schichten. Die Modelle werden auf den zweidimensionalen binären Klassifikationsdatensätzen: `moons`, `circles` und `spiral` evaluiert. Neben der Klassifikationsgenauigkeit umfasst die Analyse die Anzahl und Flächenverteilung der Aktivierungsregionen, ihre Verbindung zur Entscheidungsgrenze, sowie die

Abstände der Datenpunkte zu Aktivierungs- und Entscheidungsgrenzen. Die Aktivierungsregionen werden mithilfe eines rasterbasierten Verfahrens approximiert. Die Zuverlässigkeit dieser Approximation wird durch einen Vergleich mit der Anzahl möglicher Aktivierungsregionen geprüft. Die drei Verlustfunktionen erzielen in den meisten Versuchskonfigurationen eine ähnliche Klassifikationsleistung, während der Datensatz `spiral` deutlich schwieriger bleibt. Zwischen den Verlustfunktionen können wir nur geringe und von der jeweiligen Situation abhängige geometrische Unterschiede beobachten. Beispielsweise weist HL in einigen Situationen einen höheren Anteil von Aktivierungsregionen auf, die auf der Entscheidungsgrenze liegen, sowie größere Flächenanteile dieser Regionen. Die Architekturtiefe und die Geometrie des Datensatzes haben jedoch einen stärkeren Einfluss auf die Struktur der Aktivierungsregionen. In unseren Experimenten bilden tiefere Netzwerke mehr Aktivierungsregionen, die kleiner sind und eine ungleichmäßigere Flächenverteilung aufweisen. Beim schwierigeren Datensatz `spiral` liegt ein größerer Anteil der Aktivierungsregionen in der Nähe der Entscheidungsgrenze. Während des Trainings stabilisiert sich die Geometrie der Aktivierungsregionen in fast allen Versuchskonfigurationen, häufig aber erst nachdem sich die Klassifikationsgenauigkeit bereits stabilisiert hat. Dies deutet darauf hin, dass sich die Aktivierungsregionen weiterhin umstrukturieren. Abschließend können wir sagen, dass die Architekturtiefe und die Schwierigkeit des Datensatzes in unseren Experimenten die dominierenden Faktoren auf die Geometrie der Aktivierungsregionen sind, während der Einfluss der Verlustfunktion vergleichsweise vernachlässigbar ist.

Acronyms

A1 architecture 1

A2 architecture 2

A3 architecture 3

CE cross-entropy

HL harmonic loss

MLP multilayer perceptron

MSE mean squared error

ReLU rectified linear unit

Contents

Abstract V

Acronyms VII

1 Introduction 1

2 Background 3

2.1 ReLU Networks and Piecewise Affine Functions . . . 3

2.1.1 Activation Patterns and Activation Regions . . . 3

2.1.2 Activation Regions vs. Linear Regions . . . 9

2.2 Loss Functions . . . 10

2.2.1 Logit-Based Losses: Cross-Entropy and Mean Squared Error . . . 10

2.2.2 Harmonic Loss and Harmonic Logit Representation . . . 11

2.3 Decision Boundaries and Activation Boundaries . . . 12

2.3.1 Decision Boundaries in Logit-Based Classifiers . . . 12

2.3.2 Decision Boundaries in Harmonic Loss Classifier . . . 13

2.3.3 Activation Boundaries and Region Stability . . . 14

3 Methods 18

3.1 Experimental Design . . . 18

3.1.1 Datasets . . . 19

3.1.2 Network Architectures . . . 19

3.1.3 Training Protocol . . . 20

3.1.4 Classification Accuracy . . . 21

3.1.5 Grid-Based Approximation of Activation Regions . . . 21

3.2 Reliability of the Grid Approximation . . . 23

3.3 Activation-Region Analysis . . . 25

3.3.1 Region Count and Growth during Training . . . 25

3.3.2 Area Geometry . . . 25

3.3.3 Decision Boundary Interaction . . . 26

3.3.4 Activation-Boundary Interaction . . . 30

4 Results 32

4.1 Classification Accuracy . . . 32

4.2 Reliability of the Grid Approximation . . . 33
4.2.1 Qualitative Evolution of Activation-Pattern Partitions . . . 34
4.3 Activation-Region Analysis . . . 36
4.3.1 Region Count and Growth during Training . . . 36
4.3.2 Area Geometry . . . 38
4.3.3 Decision-Boundary Interaction . . . 39
4.3.4 Activation Boundary Interaction . . . 46

5 Discussion **48**
5.1 Grid Approximation . . . 48
5.2 Accuracy and Geometry . . . 49
5.3 Training Dynamics . . . 49
5.4 Architecture Effects . . . 50
5.4.1 Region Capacity . . . 50
5.4.2 Region Size and Inequality . . . 50
5.4.3 Boundary Involvement . . . 51
5.5 Dataset Effects . . . 51
5.5.1 Boundary Concentration . . . 51
5.6 Loss Effects . . . 52
5.7 Stability and Robustness . . . 53
5.7.1 Activation-Boundary Distance . . . 53
5.7.2 Decision-Boundary Distance . . . 53
5.8 Limitations . . . 54

6 Conclusion **56**

List of Figures **59**

List of Tables **63**

1 Introduction

Deep neural networks with rectified linear unit (ReLU) activations are widely used in modern machine learning, but their internal representation remains difficult to interpret. This thesis studies the representation through activation regions. ReLU multilayer perceptrons are continuous piecewise-affine functions and partition the input space into these regions. However, two different networks may achieve similar classification accuracy while using different internal representations and different activation region partitions. Accuracy alone is not sufficient to describe how a trained network solves a task, since models with high standard accuracy can for example still be vulnerable to small adversarial perturbations [1, 2]. This motivates the use of various activation region metrics to evaluate their performance. Motivated by the promising results of Baek et al. [3] in achieving improved structure in embedding space by using harmonic loss (HL) for model training, we see the possibility that the choice of loss function might influence the geometry of activation regions as well. For the comparison, we use cross-entropy (CE) and mean squared error (MSE) since both are widely used in machine learning tasks and therefore serve as baselines [4, 5]. The central research question is whether these different loss functions affect the activation-region geometries in ReLU networks. Since activation-region geometry can also depend on network capacity and dataset geometry, architecture depth and dataset type are varied as controlled factors [6, 7]. This makes it possible to differentiate between geometric changes caused by the loss function and effects caused by the architecture or the classification task. The experiments of this thesis include a small (one hidden layer), a medium (two hidden layers) and a large model (three hidden layers); each of them has a layer width of five. All of the experiments are performed on three two-dimensional binary classification datasets of varying difficulty, which are simple to visualize while still requiring nonlinear decision boundaries. The analysis of this thesis is structured into several steps: First, we compare predictive performance in order to verify whether models with different loss functions differ in their activation-region geometry, while reaching similar accuracy. Then, we analyze the general geometry of the activation-region partition. Previous research on activation regions has focused primarily on the number of possible regions and their upper bounds and how they relate to expressivity of the network: Pascanu et al. [6] showed that deep networks with piecewise linear activations can create many more activation regions in theory than shallow networks, leading to exponential growth. On the other hand, Montúfar et al. [7] gave a lower bound on the maximal number of linear regions of a ReLU network. Building on this idea, we measure the number of activation regions, their growth and area distribution during training . Together, these metrics describe the development and fragmentation of the

partition. Furthermore, we study the interaction between activation regions and the decision boundary. By measuring these boundary regions, we can analyze the extent to which models cluster their expressive capacity around the decision boundary. Other research on activation regions assesses classification stability and adversarial attacks. Croce et al. [8] addressed this by measuring the distance from data points to the decision boundary and to the activation boundary. Here, the former is related to local classification stability, while the latter relates to a change in the affine regions of the network. Following this work, we compare both distances in our setting, which allows us to distinguish between stability of the final prediction and stability of the internal representation. Together, the above-mentioned analyses allow us to assess whether geometric differences between networks are primarily loss-dependent, or more strongly related to architecture depth and dataset geometry.

2 Background

In this chapter, we introduce the theoretical concepts needed for the later analysis. We first describe ReLU networks as piecewise affine functions and define activation patterns and how they relate to activation regions. We then provide a more intuitive visual interpretation of activation regions and explain the difference between activation regions and linear regions. Afterward, we introduce the three loss functions compared in this thesis: HL, MSE and CE. The final section introduces decision boundaries and activation boundaries, which are important for our later distance metrics between data points and these boundaries.

2.1 ReLU Networks and Piecewise Affine Functions

A feedforward neural network is built from layers that apply affine transformations and nonlinear activation functions. In this thesis, the hidden layers use the rectified linear unit (ReLU),

$$\mathrm{ReLU}(z) = \max(0, z). \tag{2.1}$$

The ReLU is piecewise linear: it equals z for $z > 0$ and 0 for $z \leq 0$ [2, Ch. 6]. A neural network with affine layers and ReLU activations represents a continuous function that is affine on different regions of the input space [6]. This piecewise-affine structure motivates the central concepts of our analysis: activation patterns and activation regions, which are introduced in the following section.

2.1.1 Activation Patterns and Activation Regions

The property that a ReLU network can be seen as a piecewise-affine function leads to a partition in the input space of the network into regions in which the network behaves as an affine function. These regions within the ReLU network can be described through activation patterns [6, 7]. To get to these concepts we first consider a network with one hidden ReLU layer,

$$h(x) = \mathrm{ReLU}(Wx + b), \tag{2.2}$$

where $x \in \mathbb{R}^D$, $h(x) \in \mathbb{R}^K$ and $W \in \mathbb{R}^{K \times D}$ is the weight matrix of the hidden layer and $b \in \mathbb{R}^K$ is its bias vector. Here, K is the number of hidden neurons and D is the input dimension. For $k \in \{1, \ldots, K\}$, the preactivation of the k-th neuron is the affine function $z_k : \mathbb{R}^D \to \mathbb{R}$ defined by

$$z_k(x) = w_k^\top x + b_k, \tag{2.3}$$

where $w_k^\top$ is the k-th row of W, and $b_k \in \mathbb{R}$ is the k-th component of the bias vector b.

For an input x, the activation state of neuron k is defined by

$$s_k(x) = \mathbf{1}\{z_k(x) > 0\}. \tag{2.4}$$

$\mathbf{1}\{\cdot\}$ is the indicator function, which is 1 if the condition inside the braces is true and 0 otherwise. $s_k(x) = 1$ exactly when the k-th neuron is active (when its preactivation is strictly positive) and $s_k(x) = 0$ otherwise. For a given input x, we group the activation states into the binary vector

$$s(x) = \big(s_1(x), \ldots, s_K(x)\big) \in \{0,1\}^K, \tag{2.5}$$

which we call the activation pattern of x. We use $s \in \{0,1\}^K$ to describe a fixed activation pattern. We call the region in input space produced by a fixed activation pattern an activation region [9, 10]. Activation regions can also be seen as the preimages in input space of these activation patterns [9]. For any input x, its activation pattern $s(x)$ is obtained by a single forward pass: each hidden preactivation $z_k(x)$ is evaluated and thresholded at zero as in eq. (2.4), and x then lies in the corresponding activation region. So we can visualize them as a partition laid over the input space, as shown in fig. 2.1.

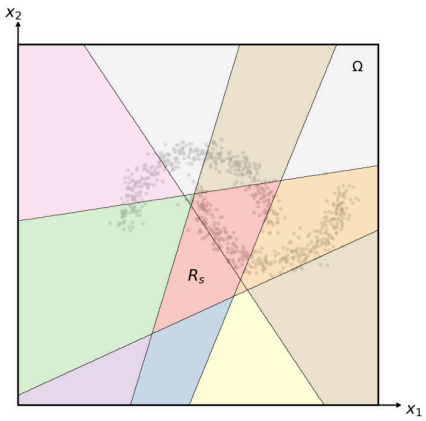


Figure 2.1: Schematic activation regions in a two-dimensional input space. Each colored polygon represents one activation region. The black edges indicate activation boundaries, across which at least one neuron changes its activation state. *Source: author's own illustration.*

The definition of points on activation boundaries differs across the literature [11]; following Hanin et al. [9], we use the convention that a neuron is active when its preactivation is strictly positive and inactive otherwise [10]. The activation region can now be described for a fixed activation pattern candidate $s \in \{0,1\}^K$ by

$$R_s = \left\{x \in \mathbb{R}^D : z_k(x) > 0 \text{ if } s_k = 1, \quad z_k(x) \le 0 \text{ if } s_k = 0\right\}. \tag{2.6}$$

Because zero preactivations are assigned to the inactive state, R_s is generally neither open nor closed. But for computational purposes, we also consider the closed cell, that is, the closed version of the region R_s. The closed cell uses a non-strict sign definition for the case of $z_k(x) = 0$, therefore we can define

$$\overline{R}_s = \left\{ x \in \mathbb{R}^D : z_k(x) \geq 0 \text{ if } s_k = 1, \quad z_k(x) \leq 0 \text{ if } s_k = 0 \right\}. \tag{2.7}$$

The closed-cell form defines feasibility using non-strict linear inequalities and is therefore directly compatible with standard linear-programming formulations [12]. We use this later in section 3.2. Throughout this thesis, the term *activation region* refers to the general geometric concept. When the boundary convention is relevant, we distinguish explicitly between the activation region R_s and the closed cell $\overline{R}_s$. Beyond describing which inputs fall in a region, the region also determines the network's behavior on it: inside each region the network behaves like an affine function [6, 13]. This is because the activation pattern is fixed on R_s, so every ReLU acts either as the identity function or as zero. Therefore, some network functions such as hidden-layer preactivations, the embedding-layer representation, or the logits are affine functions of the input within that region and will be used and defined later in the thesis. Let $g : \mathbb{R}^D \to \mathbb{R}^P$ be such a function, where P is its output dimension. On R_s, it can be written as

$$g(x) = M_s x + c_s, \qquad x \in R_s, \tag{2.8}$$

where $M_s \in \mathbb{R}^{P \times D}$ and $c_s \in \mathbb{R}^P$ are the affine matrix and offset associated with R_s. These coefficients may differ between activation regions, so the network is affine within each region and piecewise affine over the full input space.

Returning to the constraints for an activation region in eq. (2.7), for computation we can write the closed-cell inequalities in matrix form as:

$$\overline{R}_s = \{ x \in \mathbb{R}^D : A_s x \leq \eta_s \}. \tag{2.9}$$

For the one-hidden-layer network in eq. (2.2), $A_s \in \mathbb{R}^{K \times D}$ and $\eta_s \in \mathbb{R}^K$ are defined rowwise by

$$(A_s)_k = (1 - 2s_k) w_k^\top, \qquad (\eta_s)_k = (2s_k - 1) b_k. \tag{2.10}$$

The k-th inequality is $-w_k^\top x \leq b_k$ when $s_k = 1$, and $w_k^\top x \leq -b_k$ when $s_k = 0$. Each activation region is therefore a convex set, being an intersection of finitely many half-spaces [14]. For deeper networks, we can extend the same construction now. Each hidden preactivation is an affine function of the input, computed recursively through the layers [10]. Fixing s, the hidden

representation after layer $\ell - 1$ is

$$h^{(\ell-1)}(x) = M_s^{(\ell-1)} x + c_s^{(\ell-1)}, \tag{2.11}$$

where $M_s^{(\ell-1)}, c_s^{(\ell-1)}$ are the affine matrix and offset of the layer-$(\ell-1)$ representation on R_s (a special case of the general affine-region function in eq. (2.8)). The layer-ℓ preactivations are then again affine in x,

$$z^{(\ell)}(x) = W^{(\ell)} M_s^{(\ell-1)} x + W^{(\ell)} c_s^{(\ell-1)} + b^{(\ell)}, \tag{2.12}$$

and applying the fixed activation pattern of layer ℓ gives $M_s^{(\ell)}, c_s^{(\ell)}$, so the recursion continues through all layers. After the recursion, every hidden neuron k has an affine preactivation on R_s,

$$z_k(x) = \alpha_{s,k}^\top x + \beta_{s,k}, \tag{2.13}$$

with $\alpha_{s,k} \in \mathbb{R}^D$ and $\beta_{s,k} \in \mathbb{R}$, whose coefficients we can take from the row of the affine function just computed. Each neuron contributes one inequality with the sign convention of eq. (2.10): row k has the coefficient $(A_s)_k = (1 - 2s_k)\alpha_{s,k}^\top$ and $(\eta_s)_k = (2s_k - 1)\beta_{s,k}$. Combining these rows over all hidden neurons $k \in \mathcal{K}$ gives $A_s \in \mathbb{R}^{|\mathcal{K}| \times D}$ and $\eta_s \in \mathbb{R}^{|\mathcal{K}|}$, so the description $\overline{R}_s = \{x : A_s x \leq \eta_s\}$ is valid for any depth.

If the system in eq. (2.9) is feasible, meaning that there exists at least one input $x \in \mathbb{R}^D$ satisfying all inequalities, the region is a possible candidate. This feasibility test asks whether a given candidate pattern is realized by any input, in contrast to the forward evaluation in eq. (2.5), which instead assigns an already-given point to its pattern. If the system is infeasible, the corresponding pattern cannot be realized by any input [10, 15]. Throughout this thesis, we use the term activation region only for a feasible activation cell with nonempty interior in $\mathbb{R}^D$. Lower-dimensional feasible sets are excluded from the region count [10].

Intuitive Visualization of Activation Regions

In this section we give the reader a more intuitive approach toward activation regions by visualizing the construction.

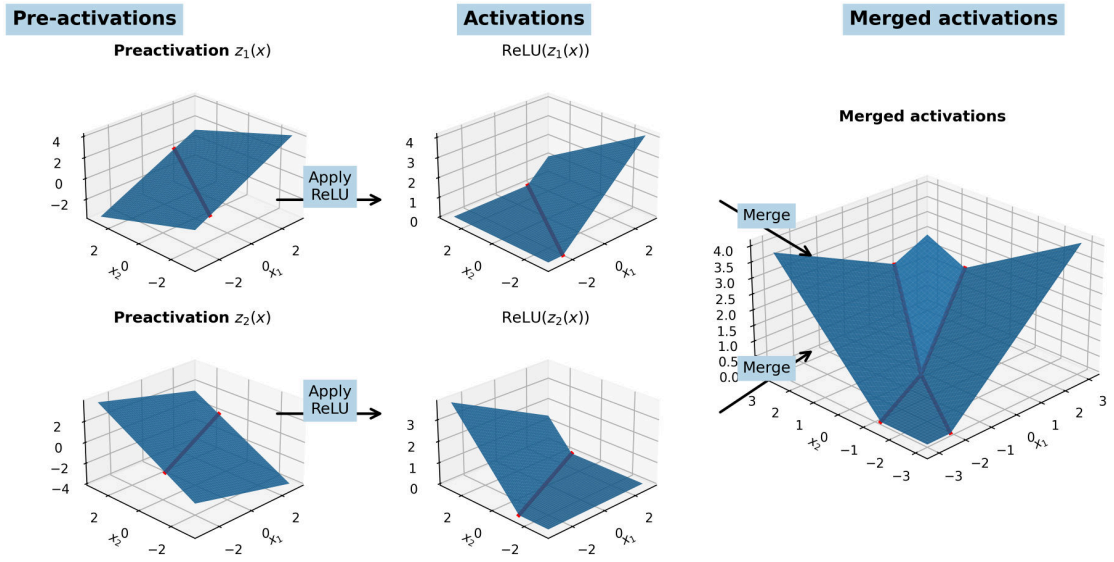


Figure 2.2: Stepwise construction of activation regions from two preactivation functions. Source: author's own illustration, adapted from Pukowski et al. [16, Fig. 1].

Figure 2.2 shows how two ReLU neurons in a single hidden layer shape the input space into activation regions. Each preactivation $z_k(x)$ is an affine plane over the two-dimensional input space, and its zero set $z_k(x) = 0$ is a line (the line where neuron k switches between active and inactive). ReLU keeps the positive side of the plane and flattens the negative side to zero, folding the surface along that line. The two lines together cut the plane into four regions, one per combination $(s_1, s_2) \in \{0, 1\}^2$, so each region is labeled by which neurons are active there. The rightmost panel shows the combined output of both neurons.

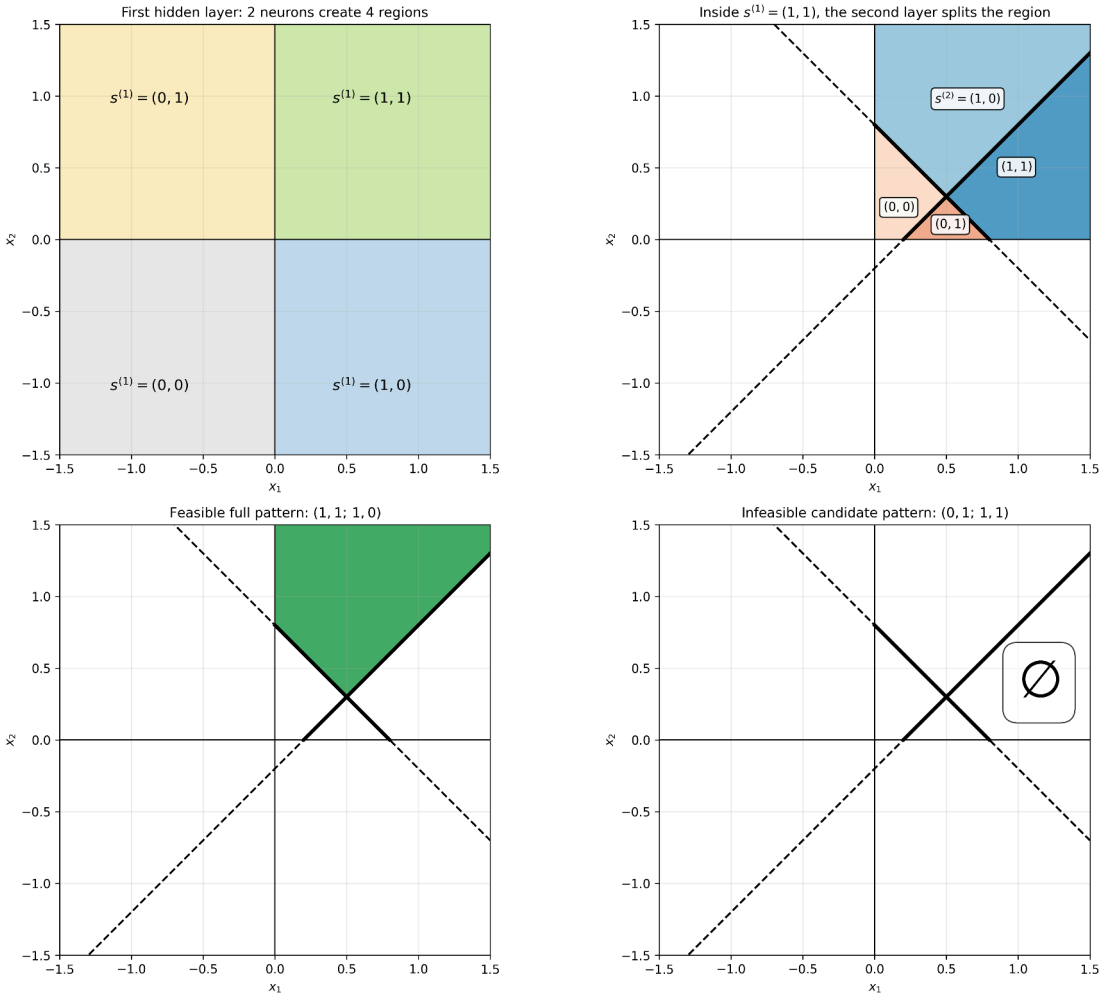


Figure 2.3: Illustration of activation regions in a simple two-dimensional ReLU network, with two hidden layers. *Source: author's own illustration.*

Figure 2.3 extends the idea of fig. 2.2 to two hidden layers of two neurons each, and shows why not every pattern is feasible. The first-layer boundaries split the plane into four cells (upper-left). Now, for the fixed pattern in the first layer $s^{(1)}$, each second-layer preactivation is affine in that parent region (upper-right); its zero set is a line inside the region, and the dashed line extensions outside are not part of the boundaries, because the affine function is only valid where the activation pattern holds [17]. An activation pattern $s = (s^{(1)}, s^{(2)})$ is feasible only when both layers' inequalities give a common solution. Both possible cases can be seen here: $s = (1, 1, 1, 0)$ gives a feasible region (lower-left panel), while $s = (0, 1, 1, 1)$ is infeasible, because its intersection is empty (lower-right panel) [10].

2.1.2 Activation Regions vs. Linear Regions

Terminology concerning activation regions and linear regions is not fully uniform across the literature [15, 18]. An activation region is defined by a fixed activation pattern, which means the activation region is a subset of the input space on which the activation pattern stays unchanged [9]. A linear region is usually defined as a subset of the input space on which the network computes the same affine function [9]. Neighboring activation regions can belong to the same linear region if they have different activation patterns but have the same underlying affine function. As a result, activation regions can form a finer partition than linear regions [9].

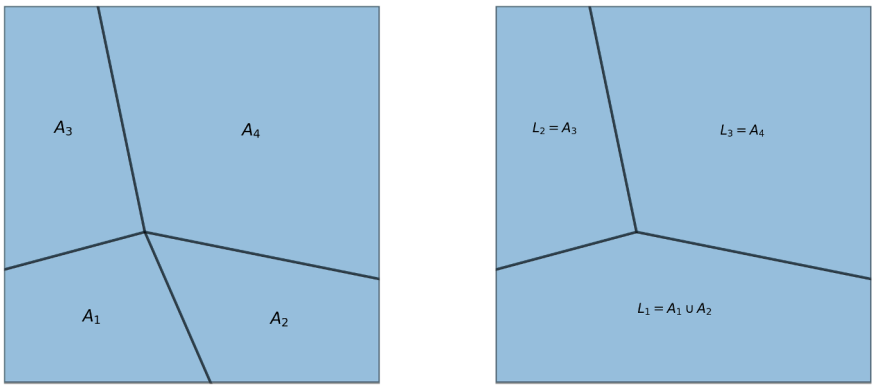


Figure 2.4: Schematic distinction between activation regions and linear regions. The activation regions A_1 and A_2 have the same affine function and therefore form the single linear region L_1. Source: author's own illustration.

Figure 2.4 illustrates the distinction between activation regions and linear regions [9]. Two neighboring activation regions may have different activation patterns and are therefore counted separately as activation regions, even if they realize the same affine function and together form one linear region.

2.2 Loss Functions

As introduced in the introduction, the choice of different loss functions can affect representation geometry even when models achieve similar classification accuracy [19]. Therefore, in this section we summarize the three loss functions used in the experiments. CE and MSE are discussed together because they use the same classifier architecture, while HL uses a different classification rule. We first introduce the notation shared by all three loss functions. Let $C = \{0, \dots, C-1\}$ be the set of class indices, where C is the number of classes. For a training example (x, y), let $x \in \mathbb{R}^D$ be the network input and $y \in C$ its class label. Let $\phi(x) \in \mathbb{R}^{d_{\text{emb}}}$ be the embedding-layer representation, where d_{emb} is the embedding dimension. All three models use the representation $\phi(x)$, but they differ in how the embeddings are converted into class probabilities. The CE and MSE models use an affine output layer followed by softmax and the HL model uses distances to learned class prototypes followed by the harmonic maximum.

2.2.1 Logit-Based Losses: Cross-Entropy and Mean Squared Error

For the logit-based classifiers used with CE and MSE, let $W_{\text{out}} \in \mathbb{R}^{d_{\text{emb}} \times C}$ and $b_{\text{out}} \in \mathbb{R}^C$ be the weight matrix and bias vector of the output layer. We define the one-hot target vector $e_y \in \{0,1\}^C$, which has value 1 at the position of the true class y and 0 at all other positions. Now the logit vector $f(x) \in \mathbb{R}^C$ is defined as [3]:

$$f(x) = W_{\text{out}}^\top \phi(x) + b_{\text{out}}, \tag{2.14}$$

Its c-th component $f_c(x)$ is the logit score assigned to class c. The logits are then converted into class probabilities using the softmax function [2, Ch. 6]. For each $c \in C$, we define

$$p_c^{\text{soft}}(x) = \frac{\exp\left(f_c(x)\right)}{\sum_{j \in C} \exp\left(f_j(x)\right)}. \tag{2.15}$$

The complete probability vector is then defined by

$$p^{\text{soft}}(x) = \left(p_0^{\text{soft}}(x), \dots, p_{C-1}^{\text{soft}}(x)\right) \in [0,1]^C. \tag{2.16}$$

Cross-entropy and mean squared error are therefore both applied to the same logit-based classifier and differ only in how the predicted probabilities are penalized.

Cross Entropy

The CE loss is the negative log-probability assigned to the correct class [2, Ch. 6, Sec. 6.2]:

$$\mathcal{L}_{\mathrm{CE}}(x, y) = -\sum_{c=0}^{C-1} (e_y)_c \log p_c^{\mathrm{soft}}(x) = -\log p_y^{\mathrm{soft}}(x). \tag{2.17}$$

Mean Squared Error

The mean squared error, also called the squared loss or L_2 loss, measures the average squared difference between predicted and target values [5]. We adapt this general definition to our setting by applying the loss to the softmax probability vector $p^{\mathrm{soft}}(x) \in \mathbb{R}^C$ and the one-hot target vector $e_y \in \{0, 1\}^C$ defined above. The loss used in this thesis is therefore

$$\mathcal{L}_{\mathrm{MSE}}(x, y) = \frac{1}{C} \left\| p^{\mathrm{soft}}(x) - e_y \right\|_2^2, \tag{2.18}$$

where $\| \cdot \|_2$ is the Euclidean norm.

We note that CE and MSE use the same logit-based classifier and the same softmax probabilities. They only differ in the applied penalty: CE applies a logarithmic penalty to the probability difference, whereas MSE applies a squared-error penalty to the probability vector.

2.2.2 Harmonic Loss and Harmonic Logit Representation

The HL model uses the same network architecture up to the embedding layer $\phi(x)$, but replaces the affine output layer and softmax classifier with a different classification rule. For each class, harmonic loss learns a class prototype μ_c in the embedding space. The corresponding class probabilities are then calculated from the distances between the embedding and the prototypes [3]. Baek et al. [3] interpret these prototypes as class centers.

For each class $c \in \{0, \ldots, C-1\}$, the distance between the embedding $\phi(x)$ and class prototype $\mu_c \in \mathbb{R}^{d_{\mathrm{emb}}}$ is used to define the harmonic logit, defined by Baek et al. [3]:

$$r_c(x) = \|\phi(x) - \mu_c\|_2. \tag{2.19}$$

To receive the class probabilities, we use the harmonic maximum (HarMax), which is an alternative to the softmax function introduced by Baek et al. [3, Sec. 2].

$$p_c^{\mathrm{harm}}(x) = \frac{r_c(x)^{-q}}{\sum_{j=0}^{C-1} r_j(x)^{-q}}. \tag{2.20}$$

Here, $q > 0$ is the harmonic exponent, $p_c^{\mathrm{harm}}(x)$ is the probability given to class c, and j indexes

the class prototypes.

HL can now be defined as the negative log-probability given to the correct class:

$$\mathcal{L}_{\text{HL}}(x, y) = -\log p_y^{\text{harm}}(x). \tag{2.21}$$

HL and CE both use the negative logarithm of the probability for the true class, but differ in how those probabilities are constructed: CE uses conventional logits followed by softmax, while HL uses distances to class prototypes followed by HarMax [3].

2.3 Decision Boundaries and Activation Boundaries

Now we look at two related but distinct types of boundaries in our ReLU networks. The first is the decision boundary, where the predicted class changes. The second is the activation boundary, where at least one ReLU unit changes its activation state, and we move from one activation region to another in input space. Distances to these boundaries can be related to different forms of network stability [8]. The distance to the decision boundary is the smallest class-changing perturbation, while the distance to the nearest activation boundary is the smallest perturbation that changes the activation pattern. Both distances are illustrated in fig. 2.5.

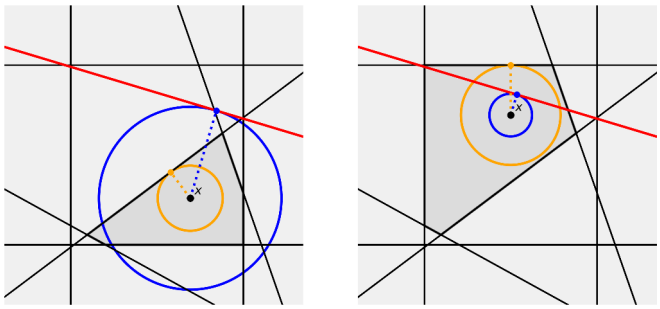


Figure 2.5: Distances to activation and decision boundaries in a ReLU network. The blue line is distance to the activation boundary and the orange line to the decision boundary. *Source: author's own illustration, adapted from Croce et al. [8, Fig. 1].*

2.3.1 Decision Boundaries in Logit-Based Classifiers

For a binary classifier with logits, $f_0(x)$ and $f_1(x)$, the predicted class is the one with the larger score. Therefore, the boundary between the two classes is defined by setting the two scores equal, $f_0(x) = f_1(x)$, where f_c is the logit score assigned to class c [20]. We define the boundary by using a margin

$$m(x) = f_1(x) - f_0(x). \tag{2.22}$$

The sign of this margin gives the predicted class. Therefore, the decision boundary is then:

$$\mathcal{B} = \{x \in \mathbb{R}^D : m(x) = 0\}, \tag{2.23}$$

The zero level set of the logit margin is equivalent to the set where the two softmax probabilities are equal, so we get

$$p_0^{\text{soft}}(x) = p_1^{\text{soft}}(x) \quad \Longleftrightarrow \quad f_0(x) = f_1(x). \tag{2.24}$$

We use the logit margin because it is piecewise affine, while the softmax probabilities are nonlinear functions of the logits [2, 20]. As described in Section 2.1.1, each activation region $R_s \subseteq \mathbb{R}^D$ corresponds to a fixed activation pattern s. Inside the region, the network function is affine,

$$f(x) = M_s x + c_s, \qquad x \in R_s, \tag{2.25}$$

where, for a binary classifier, we have $M_s \in \mathbb{R}^{2\times D}$ and $c_s \in \mathbb{R}^2$. The signed logit margin is therefore also affine on R_s:

$$m(x) = f_1(x) - f_0(x) = \alpha_s^\top x + \beta_s, \qquad x \in R_s. \tag{2.26}$$

Here, $\alpha_s \in \mathbb{R}^D$ and $\beta_s \in \mathbb{R}$ are the coefficients of the logit margin on R_s, with $\alpha_s = (M_s)_1 - (M_s)_0$ and $\beta_s = (c_s)_1 - (c_s)_0$. The part of the decision boundary inside this activation region is then defined as

$$\mathcal{B} \cap R_s = \{x \in R_s : \alpha_s^\top x + \beta_s = 0\}. \tag{2.27}$$

If $\alpha_s \neq 0$ and the zero set intersects R_s, the local decision boundary is an $(D-1)$-dimensional hyperplane, but only valid inside R_s. If $\alpha_s = 0$, the margin is constant on the region, so the zero set is either empty or there is no solution inside region R_s and therefore the boundary does not cross through this region. The global decision boundary can then be constructed by the union of these local hyperplanes [18, 21, 22].

2.3.2 Decision Boundaries in Harmonic Loss Classifier

The logit-based description above applies to the CE and MSE models. For HL, as explained in Section 2.2.2, the classifier assigns class probabilities according to distances between the embedding $\phi(x)$ and class prototypes μ_c. Hence, the predicted class is determined by which prototype is closest in embedding space [3]. In the binary case, the decision boundary is therefore the set of embeddings for which both class prototypes are equally close [3]:

$$\|\phi(x) - \mu_0\|_2 = \|\phi(x) - \mu_1\|_2. \tag{2.28}$$

To make the harmonic classifier comparable to the logit-based classifiers, we use the squared-distance as the harmonic margin,

$$\|\phi(x) - \mu_0\|_2^2 - \|\phi(x) - \mu_1\|_2^2 = 0. \tag{2.29}$$

Squaring is strictly monotone on the non-negative reals and both distances are non-negative, so the squared-distance equation has the same solution set as eq. (2.28) and therefore does not change the position of the decision boundary. Expanding the equation gives:

$$\begin{aligned} m_{\text{harm}}(x) &= \|\phi(x) - \mu_0\|_2^2 - \|\phi(x) - \mu_1\|_2^2 \\ &= 2(\mu_1 - \mu_0)^\top \phi(x) + \|\mu_0\|_2^2 - \|\mu_1\|_2^2. \end{aligned} \tag{2.30}$$

In embedding space, the decision boundary for HL is the hyperplane between the two prototypes. The expansion in eq. (2.30) shows that the quadratic terms $\|\phi(x)\|_2^2$ cancel, so m_{harm} is affine in the embedding $\phi(x)$. Since ϕ is affine on each activation region R_s (eq. (2.8)), m_{harm} is piecewise affine in the input x, exactly like the logit margin of eq. (2.26). It follows that the decision boundary is the preimage of this hyperplane in input space under the embedding map.

2.3.3 Activation Boundaries and Region Stability

The following definitions largely follow Croce et al. [8] but are adapted in notation for our purposes. Each activation region has a boundary where its activation pattern transitions to another state. In this subsection, $k \in \mathcal{K}$ is a global index over all hidden ReLU neurons, including neurons from different hidden layers. For a hidden neuron with preactivation $z_k(u)$, the activation boundary is the set of input points where this preactivation is zero,

$$\mathcal{Z}_k = \{u \in \mathbb{R}^D : z_k(u) = 0\}. \tag{2.31}$$

Here, $u \in \mathbb{R}^D$ is an arbitrary input point, $z_k(u)$ is the preactivation of hidden neuron k, and $\mathcal{Z}_k$ is the zero set of this preactivation. Within a fixed activation region the activation boundary is a hyperplane [6, 7, 10]. In our setting, $s = s(x)$ is the fixed activation pattern for input x, which lies in activation region R_s. As introduced in eq. (2.13), each hidden preactivation is affine on R_s with coefficients $\alpha_{s,k}, \beta_{s,k}$ obtained from the recursion in eq. (2.11). Writing it here with the free input variable u,

$$z_k(u) = \alpha_{s,k}^\top u + \beta_{s,k}, \qquad u \in R_s. \tag{2.32}$$

When set to zero, the corresponding local activation boundary is

$$\mathcal{H}_{s,k} = \left\{u \in \mathbb{R}^D : \alpha_{s,k}^\top u + \beta_{s,k} = 0\right\}. \tag{2.33}$$

However, $\mathcal{H}_{s,k}$ is a local affine extension. The affine expression $z_k(u) = \alpha_{s,k}^\top u + \beta_{s,k}$ is only valid on the activation region R_s. Therefore, the global zero set of neuron k is received by taking the union only over the valid part of the affine extension in each region:

$$\mathcal{Z}_k = \bigcup_s \left(\mathcal{H}_{s,k} \cap \overline{R}_s \right). \tag{2.34}$$

The parts of $\mathcal{H}_{s,k}$ outside $\overline{R}_s$ are only affine continuations and are not part of the realized activation boundary. This representation follows the description of piecewise-affine neural networks [17, 22]. The boundary of the activation region R_s is built from the union of local activation boundaries $\mathcal{H}_{s,k}$. Hence, we can define the boundary for a specific region by

$$\partial R_s = \overline{R}_s \cap \bigcup_{k \in \mathcal{K}} \mathcal{H}_{s,k}. \tag{2.35}$$

where $\mathcal{K}$ is the set of all hidden ReLU neurons [8, 17]. Following Croce et al. [8], we define the activation-boundary distance as the distance from x to the boundary of its current activation region:

$$d_{\text{act}}(x) = \min_{v \in \partial R_s} \|v - x\|_2. \tag{2.36}$$

For each hidden neuron $k \in \mathcal{K}$ satisfying $\|\alpha_{s,k}\|_2 > 0$, let $d_{s,k}(x)$ be the shortest distance from x to the hyperplane $\mathcal{H}_{s,k}$. Using the point-to-hyperplane distance formula, this distance is

$$d_{s,k}(x) = \frac{\left| \alpha_{s,k}^\top x + \beta_{s,k} \right|}{\|\alpha_{s,k}\|_2} = \frac{|z_k(x)|}{\|\alpha_{s,k}\|_2}. \tag{2.37}$$

Neurons with $\|\alpha_{s,k}\|_2 = 0$ have a preactivation that is constant on R_s; they define no activation boundary within the region and are therefore excluded. Since $\overline{R}_s$ is a closed cell, the distance from a point $x \in \text{int}(\overline{R}_s)$ to ∂R_s is the smallest of the point-to-hyperplane distances $d_{s,k}(x)$; hyperplanes that do not touch $\overline{R}_s$ cannot be the nearest, so the minimum can be taken over all neurons $k \in \mathcal{K}$ with $\|\alpha_{s,k}\|_2 > 0$. Combining this with eq. (2.37) gives

$$d_{\text{act}}(x) = \min_{k \in \mathcal{K}} \frac{|z_k(x)|}{\|\alpha_{s,k}\|_2}, \qquad s = s(x). \tag{2.38}$$

where only neurons with $\|\alpha_{s,k}\|_2 > 0$ are included in $\mathcal{K}$. Equation (2.38) is the Euclidean activation-boundary distance, which will be used in our experiments. It measures the smallest perturbation required to bring at least one preactivation to zero, that is, to reach the boundary of the current activation region. An arbitrarily small additional perturbation across that boundary may then change the activation state.

Visualization of Activation Boundary Construction

To make the preceding construction more intuitive, we illustrate it for a simple two-hidden-layer ReLU network with two neurons in each hidden layer.

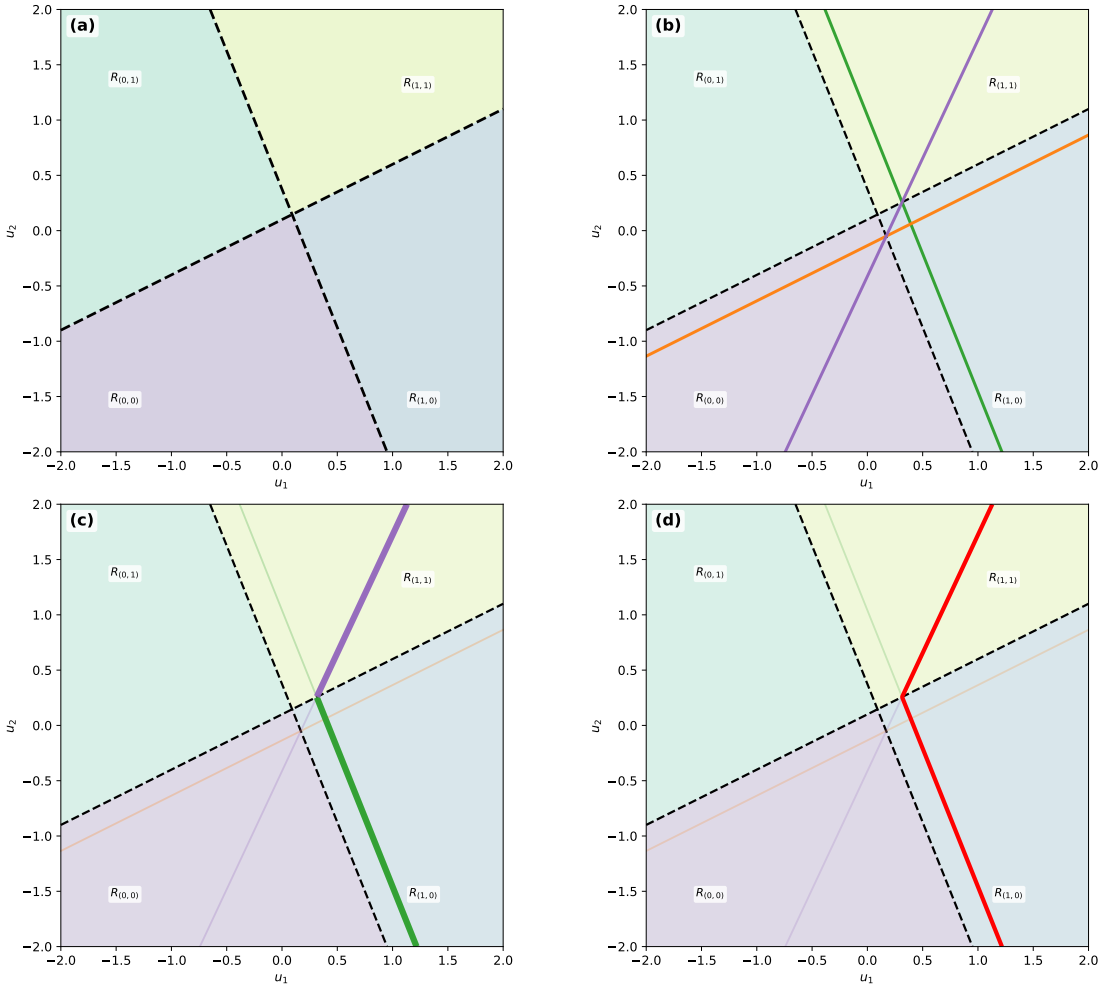


Figure 2.6: Illustration of first-layer activation regions and the zero set of a second-layer ReLU preactivation. *Source: author's own illustration, based on Lim et al. [23, Sec. 3.2 and Fig. 2].*

Figure 2.6 builds the zero set of a single second-layer neuron in four steps, showing why a deep neuron's boundary is made of pieces rather than one straight line. Panel (a): the two first-layer hyperplanes split the input into four regions $R^{(1)}_{(0,0)}, R^{(1)}_{(1,0)}, R^{(1)}_{(0,1)}, R^{(1)}_{(1,1)}$, on each the second-layer neuron is a different affine function [23]. Panel (b): each of those affine functions of the second layer neuron is extended across the whole plane, giving four full zero lines. Panel (c) is the key step, where only the segment of each line lying inside its own parent region is kept;

the rest is an affine extension with no meaning outside that region [17, 22]. Panel (d): the valid segments are joined into the global zero set

$$\mathcal{Z}_k = \{u : z_k(u) = 0\}.$$

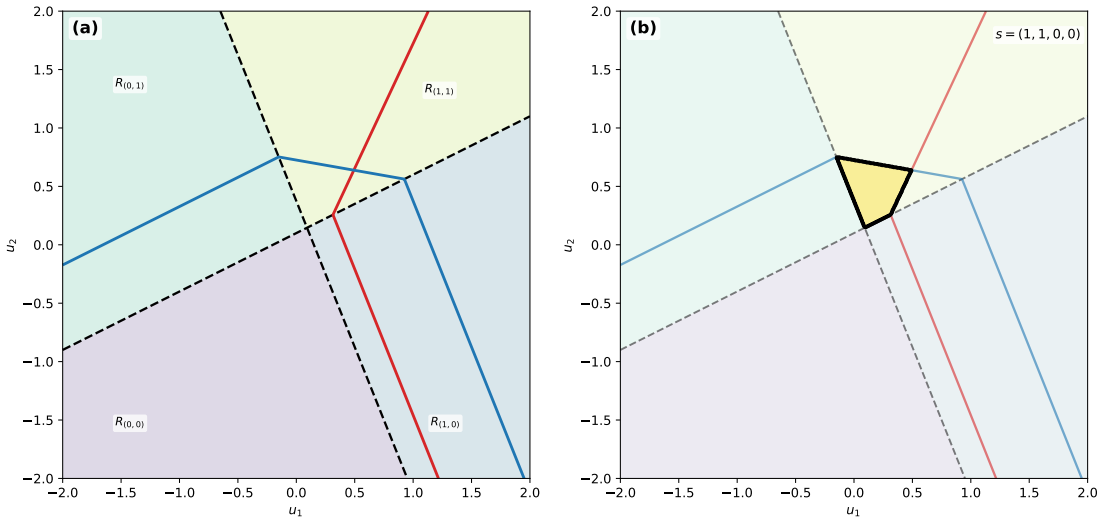


Figure 2.7: Illustration of the activation boundaries of two second-layer ReLU neurons in the left panel and the boundary of one chosen activation region in the right panel. The black outline is its boundary ∂R_s. *Source: author's own illustration.*

Figure 2.7 shows how those per-neuron zero sets are combined to form the boundary of one region. Panel (a) shows the first-layer activation regions together with the zero level sets of two second-layer neurons. In Panel (b) we then choose a single region $\overline{R}_s$ for the pattern $s = (1, 1; 0, 0)$. As in eq. (2.35), that boundary is where $\overline{R}_s$ meets the zero hyperplanes of the hidden neurons. So, $\mathcal{Z}_k$ is neuron-specific (left panel), while ∂R_s is region-specific (right panel).

3 Methods

After introducing ReLU networks, activation regions and loss functions, we now describe how these components are combined in our experiments to analyze how different training factors shape activation-region geometry. We first introduce the experimental design: datasets, architectures, and training factors. Then, we define the grid approximation for constructing the activation regions used in our experiments. Afterward, we assess the reliability and interpretation of the grid approximation. Finally, we define the main metrics with which we evaluate each model at several training checkpoints: the number of activation regions, region-size metrics, decision-boundary metrics, and activation-boundary metrics.

3.1 Experimental Design

As stated in chapter 1, we use models trained with different loss functions, namely CE, MSE, and HL, to compare how they influence the geometry of activation regions.

Table 3.1: Overview of experimental factors.

Factor	Values
Datasets	`moons`, `circles`, `spiral`
Loss functions	Cross-entropy, mean squared error, harmonic loss
Architectures	$A1$, $A2$, $A3$
Embedding dimension	$d_{\text{emb}} = 2$
Random seeds	10
Training epochs	4000
Checkpoints	25, 50, 100, 250, 500, 1000, 2000, 3000, 4000
Grid resolution	800×800

Table 3.1 summarizes the experimental setup, where we vary four factors: the dataset, the loss function, the network architecture, and the random seed used for model initialization. For every combination of these factors, one model is trained. To isolate the effect of the loss function, other factors are controlled by using the same train-test splits, checkpoint epochs, and random seeds across all models. We do not use cross-validation because of the computational limitations of this thesis. In the following section, each factor is discussed separately.

3.1.1 Datasets

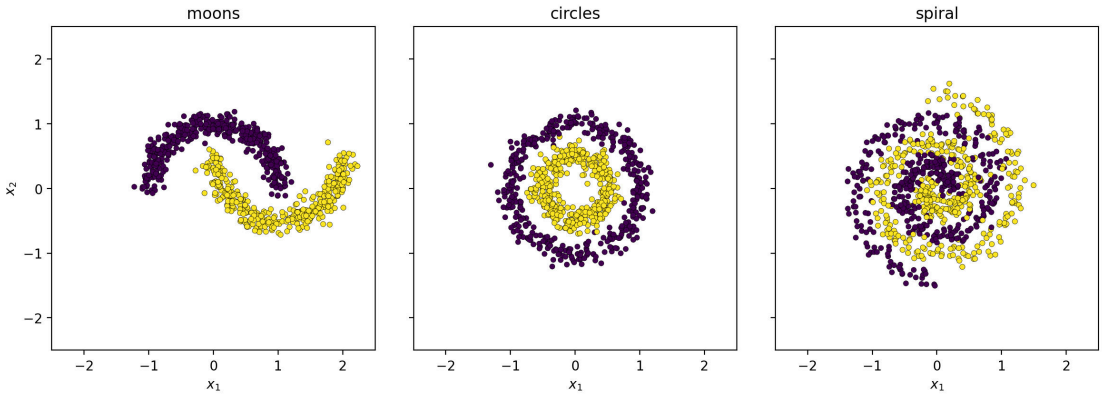


Figure 3.1: Visualization of the three two-dimensional binary classification datasets used in the experiments.

We use three two-dimensional binary classification datasets, visualized in fig. 3.1: `moons`, `circles`, and `spiral`. These datasets are simple enough for small networks used in our experiment, but they differ in geometric complexity. The `moons` and `circles` datasets are generated using the scikit-learn functions [24]. The `moons` dataset consists of two interleaving half-circles, while the `circles` dataset consists of one class arranged around an inner circle and the other around an outer circle. For `circles`, we use an inner circle factor of 0.5. The `spiral` dataset is generated manually and is adapted from the classic two-spirals classification task, in which the two classes form intertwined spiral arms [25]. Each dataset contains 800 samples in $\mathbb{R}^2$, with additive noise of standard deviation 0.10. Of these samples, 80% are used for training and 20% for testing, resulting in 640 training samples and 160 test samples per dataset.

3.1.2 Network Architectures

All models are fully connected ReLU multilayer perceptrons. The models take the two-dimensional dataset samples as input. We consider three architecture variants, shown in fig. 3.2. Architecture 1 (A1) is a network with one hidden layer of width five, architecture 2 (A2) has two hidden layers of width five, and architecture 3 (A3) has three hidden layers of width five. The hidden layers are followed by an embedding layer and then by the output layer. The architectures are intentionally kept small because the analysis requires dense grid evaluation and feasibility computations do not scale efficiently to large architectures. Width five provides enough neurons to create non-trivial activation-region structure, while still keeping the number of possible activation patterns computationally manageable. Varying the depth from one to three hidden layers allows us to analyze how increasing network complexity affects the activation-region

geometry without changing the width.

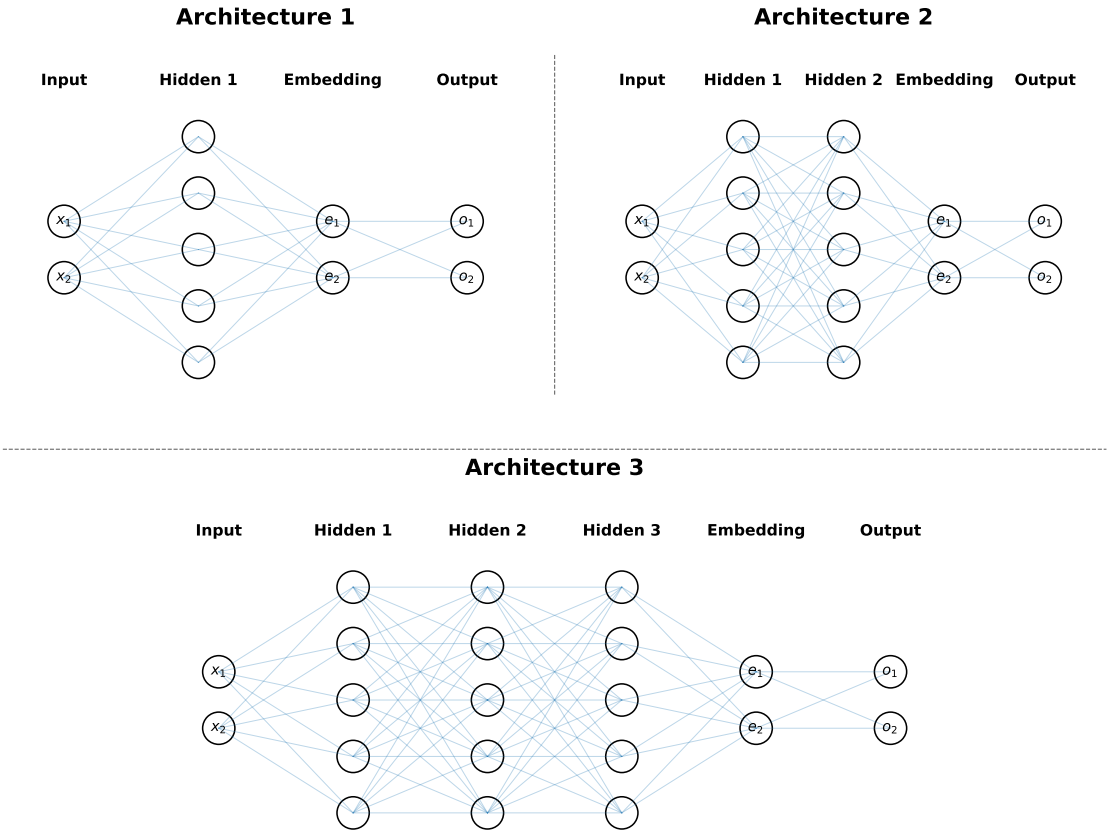


Figure 3.2: Visualization of the three architecture variants. Source: author's own illustration.

All architectures map an input $x \in \mathbb{R}^2$ to a two-dimensional embedding $\phi(x) \in \mathbb{R}^2$. The CE and MSE models use the logit-based classifier defined in section 2.2.1, while the HL models use the harmonic classifier defined in section 2.2.2. For HL, we set the harmonic exponent to $q = 2$, resulting in the inverse-squared distance weights in eq. (2.20).

3.1.3 Training Protocol

All models are trained with the same optimization protocol, independent of the loss function. We use full-batch AdamW (defined by Loshchilov and Hutter [26]) with a learning rate of $\eta = 10^{-3}$, weight decay $\lambda_{\text{wd}} = 10^{-3}$, and a cosine annealing learning-rate schedule over all 4000 training epochs. Hidden-layer weights are initialized with Kaiming initialization [27], which is appropriate for ReLU networks, while the embedding layer and the linear classifier head are initialized with Xavier initialization [28]. Biases are initialized to zero. For HL, the class prototypes are initialized from a normal distribution with mean 0 and standard deviation

0.5. This shared training protocol is used to keep training fixed, so that observed differences primarily come from the loss formulation and architecture choice rather than from different training settings. The following checkpoint epochs are used:

$$\mathcal{T}_{\text{ckpt}} = \underbrace{\{25,\ 50,\ 100,\ 250,\ 500,\ 1000,\ 2000,\ 3000,\ 4000\}}_{\text{evaluation checkpoints}}. \tag{3.1}$$

At each checkpoint, the model is evaluated using the set of metrics defined later in this chapter. For activation region metrics, we use the median across 10 seeds together with the 25th to 75th percentile band. We chose these statistics because our measured quantities are often discrete, bounded, or sensitive to rare seed-specific outliers. Therefore, median and interquartile range give a summary of the median trained model while reducing the influence of individual outliers, consistent with robust data-analysis practice [29].

3.1.4 Classification Accuracy

At every checkpoint, we report classification accuracy on the test set. For a dataset $\mathcal{D}$ with n samples $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, accuracy is defined as

$$\text{Acc}(\mathcal{D}) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{\hat{y}(x_i) = y_i\}, \tag{3.2}$$

where $\hat{y}(x_i)$ is the class predicted by the model.

3.1.5 Grid-Based Approximation of Activation Regions

Since the exact enumeration of all activation regions becomes computationally infeasible for deeper network architectures, we approximate the partition by sampling points on a grid in the input space. This also helps us to construct a geometric visualization of the network partition later. For all grid-based metrics, the input space is restricted to the bounded box in input space

$$\Omega = [-2.5, 2.5] \times [-2.5, 2.5]. \tag{3.3}$$

This box is chosen to be large enough to contain the data from all three two-dimensional datasets with additional margin around the data. At the same time, it avoids evaluating activation regions far away from the data distribution, where the activation region partition is less relevant for the classification task. We use a resolution of $r = 800$ grid points along each axis:

$$r = 800. \tag{3.4}$$

This choice is based on the grid-approximation analysis in section 4.2, which showed that $r = 800$ was the most reliable of the tested resolutions. For further metric definitions, we define the metric grid as

$$\mathcal{G} = \{g_{uv} \in \Omega : u, v = 1, \ldots, r\}, \qquad |\mathcal{G}| = r^2, \tag{3.5}$$

where g_{uv} is the grid point at horizontal index u and vertical index v, and r is the resolution along each axis. Activation patterns on the grid are computed using the strict-positive activation-state definition in eq. (2.4).

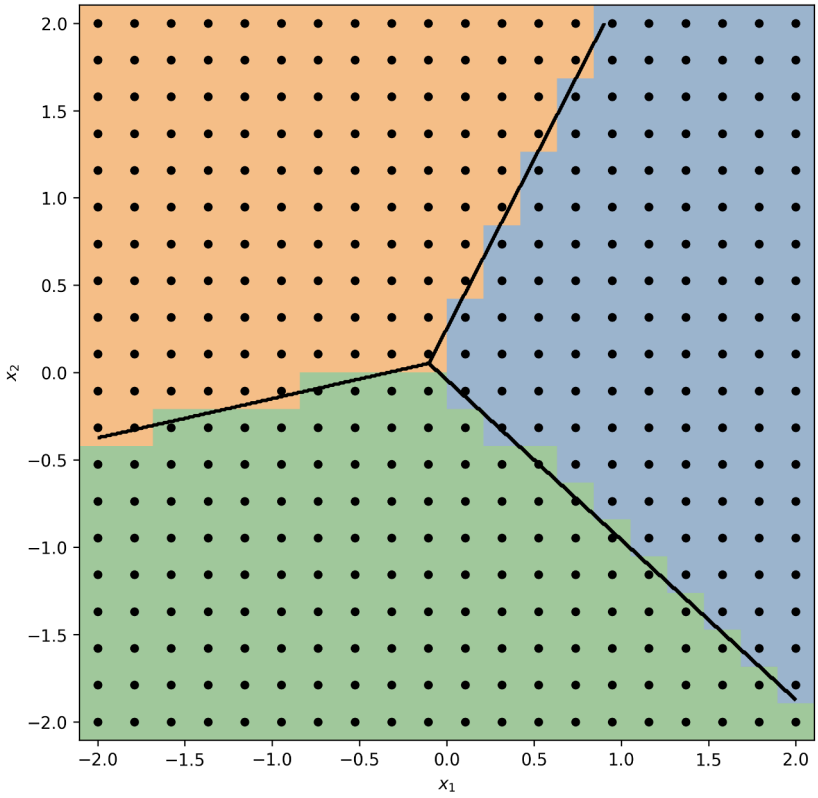


Figure 3.3: Grid-based construction of sampled activation regions from activation-pattern IDs.

Figure 3.3 shows that different activation patterns on the grid are treated as activation regions. Throughout this chapter, $R_i \subseteq \mathcal{G}$ is the i-th sampled activation region, that is, the set of all grid points with the same activation pattern:

$$R_i = \left\{g \in \mathcal{G} : s(g) = s^i\right\}, \qquad i = 1, \ldots, N, \tag{3.6}$$

where $s^i \in \{0, 1\}^{|\mathcal{K}|}$ is the activation pattern associated with region R_i. We define the set of all

activation regions by

$$\mathcal{R} = \{R_1, \ldots, R_N\}, \qquad |\mathcal{R}| = N. \tag{3.7}$$

Here, N is the number of different activation patterns on the grid.

3.2 Reliability of the Grid Approximation

The grid-based approximation has an intrinsic error: because the grid is a finite discretization of a continuous input space, it can miss very small activation regions (fig. 3.4). Any conclusion drawn from the grid metrics is only as reliable as this approximation, so we quantify the error before using it. We therefore compare the grid-based region count with a more rigorous linear-programming-based count of feasible activation regions [15].

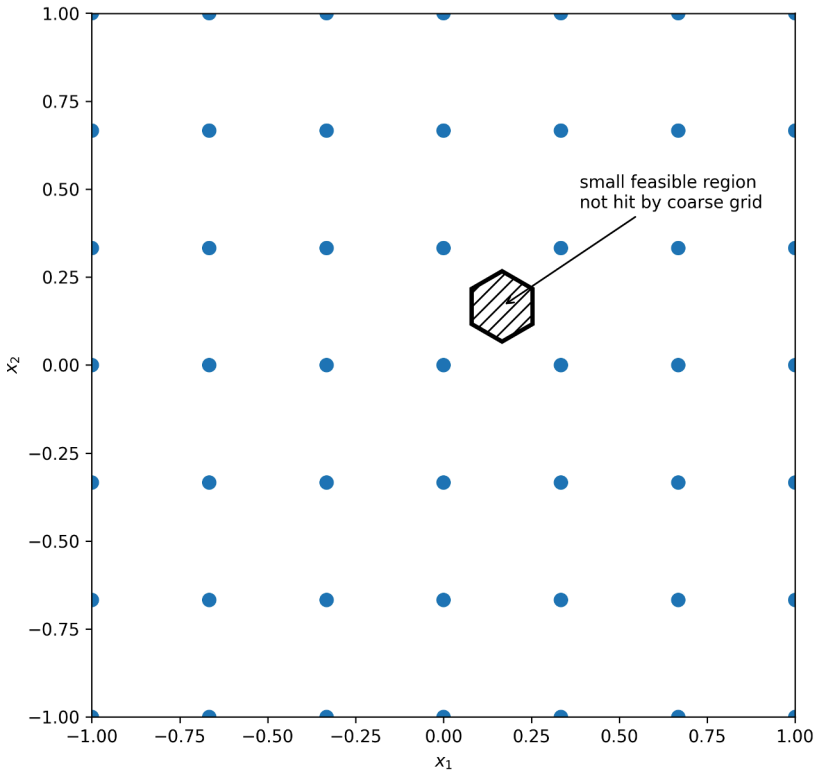


Figure 3.4: A small feasible activation region can be missed by a coarse grid.

Using the activation-pattern definition in eq. (2.5), we enumerate all activation patterns $s \in \{0,1\}^{|\mathcal{K}|}$, where $\mathcal{K}$ is the set of all hidden ReLU neurons. So, the total number of candidate activation patterns is $2^{|\mathcal{K}|}$. The number of possible candidate patterns grows exponentially as

$$2^{|\mathcal{K}|}. \tag{3.8}$$

For the three architecture variants considered in this work, this gives us $2^5 = 32$ possible activation patterns for A1, $2^{10} = 1024$ for A2, and $2^{15} = 32768$ for A3. For each candidate activation pattern, we solve the resulting linear inequality system within the bounded area Ω. Following Stargalla et al. [11], only activation regions with nonempty interior in the input space are counted. To test this condition, we maximize the radius of a Euclidean ball contained in the tested region using the Chebyshev-center formulation [14]. Candidate regions whose optimal radius does not exceed 10^{-9} are excluded from the region count. Because the number of candidate patterns grows exponentially with the number of hidden neurons, this computation is performed only for the final models at epoch 4000. For each candidate pattern s, the hidden preactivations are affine in the input after fixing the activation states, with coefficients obtained by the recursion in eq. (2.11). Writing neuron k's preactivation on R_s as $\alpha_{s,k}^\top x + \beta_{s,k}$, its activation state s_k gives the linear constraint

$$(1 - 2s_k)\alpha_{s,k}^\top x \leq (2s_k - 1)\beta_{s,k}. \tag{3.9}$$

Collecting the constraints of all hidden neurons gives the region system $A_s x \leq \eta_s$ (eq. (2.9)), which we intersect with the box $x \in \Omega$. To measure how the approximation improves with resolution, the trained networks are evaluated at the following grid resolutions:

$$r \in \{100,\ 200,\ 400,\ 600,\ 800\}. \tag{3.10}$$

At each resolution, the number of activation patterns observed on the grid is computed. This quantity is compared to the number of feasible activation patterns on the same input area. We then calculate the recall between grid count and feasible count:

$$Rec_{\text{grid}} = \frac{N_{\text{grid}}}{N_{feas}}, \tag{3.11}$$

where N_{grid} is the number of activation patterns on the grid and N_{feas} is the number of feasible activation patterns. A recall close to one means that the grid resolution captures nearly all feasible activation regions in the bounded area. We show the median of these recall values over all seeds in our graphics.

3.3 Activation-Region Analysis

In this section we define the metrics used to analyze activation-region geometry. Our goal is to find geometric properties that show how the activation-region partition evolves during training depending on the chosen architecture, loss function and dataset. We group these metrics into four families: region count, region size, decision boundary and activation boundary.

3.3.1 Region Count and Growth during Training

As shown in Section 2.1.1, the number of activation regions can be used as a geometric measure of the expressivity of ReLU networks. We show the final number of activation regions after training, but also how the region count evolves throughout training. The activation-region growth rate is computed between consecutive checkpoints. Let N_{t_1} and N_{t_2} be the numbers of activation regions at two checkpoints t_1 and t_2. Then

$$G_{growth} = \frac{\log N_{t_2} - \log N_{t_1}}{t_2 - t_1}. \tag{3.12}$$

Using the logarithm makes the metric describe relative growth rather than absolute growth. Values close to zero mean the activation-region count changes only weakly between checkpoints, suggesting that the geometric partition has largely stabilized.

3.3.2 Area Geometry

Two metrics describe the geometric structure of the partition: the region area fraction and the Gini coefficient. Together they capture how evenly or unevenly the input space is divided among regions.

Region Area Fraction

For each activation region R_i, we approximate its area by the fraction of grid points with the same activation pattern:

$$A_i = \frac{|R_i|}{|\mathcal{G}|} = \frac{|R_i|}{r^2}. \tag{3.13}$$

We summarize the median region size by the median area fraction

$$A_{\text{med}} = \underset{i=1,\dots,N}{\text{median}}\, A_i. \tag{3.14}$$

Since the activation regions partition the grid fully, their area fractions satisfy

$$\sum_{i=1}^{N} A_i = 1. \tag{3.15}$$

Gini Coefficient

To measure how unevenly the input space is divided among activation regions, we compute the Gini coefficient of the activation region areas A_i. Let $A_1, \ldots, A_N$ be the area fractions of the N activation regions, and let

$$A_{(1)} \leq A_{(2)} \leq \cdots \leq A_{(N)}$$

be the same values sorted in ascending order. That is, $A_{(i)}$ is the i-th smallest activation-region area fraction. We compute [30]:

$$G_{area} = \frac{2\sum_{i=1}^{N} iA_{(i)}}{N\sum_{i=1}^{N} A_{(i)}} - \frac{N+1}{N}. \tag{3.16}$$

Since the area fractions satisfy $\sum_{i=1}^{N} A_i = 1$, this gives an inequality measure over the region-size distribution. The Gini coefficient is a standard inequality measure [30]: here, a value near 0 means most activation regions have similar area, while a high value means the partition has many small regions and few large ones.

3.3.3 Decision Boundary Interaction

In the following section, we present the metrics for measuring the interaction between the decision boundary and the activation regions. We compute four different metrics that address different aspects of interaction with the decision boundary. First, we define how the decision boundary can be approximated on our grid, since this approximation is the basis for all following decision boundary metrics.

Calculation of the Decision Boundary and Distance Metric

For each model, we use the margin function defined in eqs. (2.22) and (2.30) to define the decision boundary as its zero level set, as described in eq. (2.23). We evaluate the margin at every point of the metric grid $\mathcal{G}$ from eq. (3.5) and approximate the zero level set by detecting a margin sign flip along neighboring horizontal and vertical grid edges. Let $g, g' \in \mathcal{G}$ be two neighboring grid points. Their edge is selected when

$$\mathbf{1}\{m(g) > 0\} \neq \mathbf{1}\{m(g') > 0\}\,. \tag{3.17}$$

When we find one endpoint that has a positive margin and a neighboring point with a non-positive margin, the exact point of the crossing is then approximated by linear interpolation. We define

$$\alpha = \frac{|m(g)|}{|m(g)| + |m(g')|}, \tag{3.18}$$

where $\alpha \in [0, 1]$ is the interpolation weight along the edge from g to g'. The decision-boundary point is then

$$b = g + \alpha\,(g' - g)\,. \tag{3.19}$$

Applying this procedure to all horizontal and vertical grid edges gives an approximation of the decision boundary,

$$\mathcal{B} = \{b_1, \ldots, b_M\} \subset \mathbb{R}^2, \tag{3.20}$$

where M is the number of boundary points. Distances to the decision boundary are then computed in input space. For any point $x \in \mathbb{R}^2$, we define its Euclidean distance to the decision boundary as

$$d_{dec}(x) = \min_{b \in \mathcal{B}} \|x - b\|_2\,. \tag{3.21}$$

Here, $\|\cdot\|_2$ is the Euclidean norm.

For the metrics where the side of the decision boundary is relevant, we additionally use the sign of the margin and define the signed distance

$$\tilde{d}_{dec}(x) = \operatorname{sign}(m(x)) \min_{b \in \mathcal{B}} \|x - b\|_2\,. \tag{3.22}$$

The sign indicates on which side of the decision boundary the point lies. We use this definition because adversarial perturbations are measured over distances in input space. For example, adversarial robustness is commonly studied by asking how far an input can be perturbed before the classifier changes its prediction, measured under an input-space norm [8]. This construction is shared by the following decision-boundary metrics.

Boundary Regions

The first metric measures the fraction of activation regions that cross the decision boundary. Let $\mathcal{R} = \{R_1, \ldots, R_N\}$ be the set of activation regions defined in eq. (3.7). For every activation region, we calculate the minimum and maximum signed distances to the decision boundary:

$$d_i^{\min} = \min_{x \in R_i} \tilde{d}_{dec}(x), \qquad d_i^{\max} = \max_{x \in R_i} \tilde{d}_{dec}(x). \tag{3.23}$$

The signed distances of the grid points in region R_i can now be summarized by the interval

$$[d_i^{\min}, d_i^{\max}].$$

We define an activation region as a boundary region when its signed-distance interval contains zero. The set of boundary regions is then

$$\mathcal{R}_{\text{dec}} = \left\{ R_i \in \mathcal{R} : d_i^{\min} \leq 0 \text{ and } d_i^{\max} \geq 0 \right\} \tag{3.24}$$

and the fraction of activation regions that are boundary regions is

$$F_{\text{dec}} = \frac{|\mathcal{R}_{\text{dec}}|}{|\mathcal{R}|} = \frac{|\mathcal{R}_{\text{dec}}|}{N}. \tag{3.25}$$

Boundary-Region Area

The second metric measures the fraction of the bounded input space occupied by boundary regions. While F_{dec} measures the number of activation regions that interact with the decision boundary, this metric measures how much of the input area is occupied by such regions. This distinction is relevant because a model may have many small activation regions touching the decision boundary, or a few large ones, so that a high boundary-region fraction F_{dec} can still occupy only a small input area. We use the area fractions A_i defined in eq. (3.13). Since the decision boundary crosses a region when the signed margin changes sign among the region's grid points, as described in section 2.3, we define the area fraction occupied by boundary regions as follows:

$$F_{\text{dec-area}} = \sum_{R_i \in \mathcal{R}_{\text{dec}}} A_i. \tag{3.26}$$

Decision-Boundary Band

Since the boundary-region metric in section 3.3.3 counts whether an activation region intersects the decision boundary, we can now extend this metric by constructing a band around the decision boundary, which is progressively widened by introducing a distance tolerance ϵ. For every activation region, we reuse the signed-distance interval $[d_i^{\min}, d_i^{\max}]$ defined in eq. (3.23). This construction is illustrated conceptually in fig. 3.5. We evaluate this metric for the trained networks at epoch 4000.

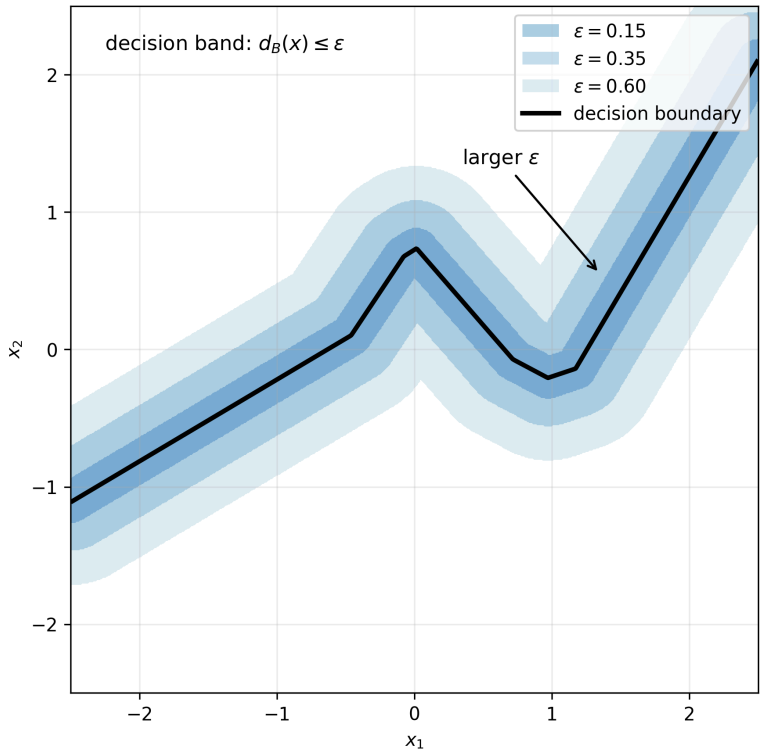


Figure 3.5: Schematic illustration of the band around the decision boundary. The band grows as the distance tolerance ϵ increases.

We define the set of activation regions lying in the boundary band as

$$\mathcal{R}_{\text{band}}(\epsilon) = \left\{ R_i \in \mathcal{R} : d_i^{\min} \leq \epsilon \text{ and } d_i^{\max} \geq -\epsilon \right\}. \tag{3.27}$$

The final metric is the fraction of activation regions belonging to this set:

$$F_{\text{band}}(\epsilon) = \frac{|\mathcal{R}_{\text{band}}(\epsilon)|}{|\mathcal{R}|} = \frac{|\mathcal{R}_{\text{band}}(\epsilon)|}{N}. \tag{3.28}$$

At $\epsilon = 0$, the band contains exactly the boundary regions, so that $\mathcal{R}_{\text{band}}(0) = \mathcal{R}_{\text{dec}}$ and therefore $F_{\text{band}}(0) = F_{\text{dec}}$. As ϵ increases, the band widens in the input space, and $F_{\text{band}}(\epsilon)$ measures how quickly additional activation regions enter the neighborhood of the decision boundary. A rapidly increasing curve indicates that many activation regions are located near the boundary.

Distance between Data Points and Decision Boundary

As a final metric, we measure the distance from each data point to the decision boundary. This metric is motivated by Croce et al. [8], in which samples farther from the decision boundary require a larger input perturbation before their predicted class might change. Let $\mathcal{D} = \{x_i\}_{i=1}^{n}$ be all training and test samples ($n = 800$), where $x_i \in \mathbb{R}^2$ is the i-th sample. For each sample $x_i \in \mathcal{D}$, we compute the Euclidean distance to the closest decision-boundary point by using Equation (3.21). We calculate the median decision-boundary distance as

$$D_{dec}^{\text{median}} = \underset{x_i \in \mathcal{D}}{\text{median}}\, d_{dec}(x_i), \tag{3.29}$$

and also report the 10th percentile as

$$D_{dec}^{0.1} = \underset{0.1}{\text{quantile}}\, \{d_{dec}(x_i) : x_i \in \mathcal{D}\}, \tag{3.30}$$

which summarizes the samples lying relatively close to the decision boundary. Larger values indicate that samples lie farther away from the decision boundary and are therefore locally more stable with respect to class-changing input perturbations [8].

3.3.4 Activation-Boundary Interaction

As introduced in Section 2.3.3, the distance to the nearest activation boundary measures how far a sample is from changing its activation pattern. We use this quantity as a local stability metric for the internal activation structure of the network [8]. This calculation is done directly in the full input space and does not use the grid approximation, since we can use eq. (2.38). For each sample $x_i \in \mathcal{D}$, we calculate its activation pattern to compute $d_{\text{act}}(x_i)$. We report the median activation-boundary distance over all training and test samples:

$$D_{\text{act}} = \underset{x_i \in \mathcal{D}}{\text{median}}\, d_{\text{act}}(x_i). \tag{3.31}$$

Smaller values of D_{act} show that the samples require smaller input perturbations to reach a change in activation pattern. But reaching an activation boundary does not necessarily change the predicted class [8].

Neuron Scaled Distance

To take differences in architecture size into account, we additionally measure

$$D_{\text{act}}^{\text{scaled}} = |\mathcal{K}|\, D_{\text{act}}, \tag{3.32}$$

where $|\mathcal{K}|$ is the number of hidden ReLU neurons. This approach follows Hanin et al. [31], who report that the distance to the nearest activation-region boundary scales approximately inversely with the number of neurons and therefore also consider their product.

4 Results

In this chapter we report the results of our experiments. We first present our general performance results and follow this with our grid robustness diagnostics. We then focus on the activation-region metrics, which include area geometry, decision-boundary metrics, and activation-boundary stability.

4.1 Classification Accuracy

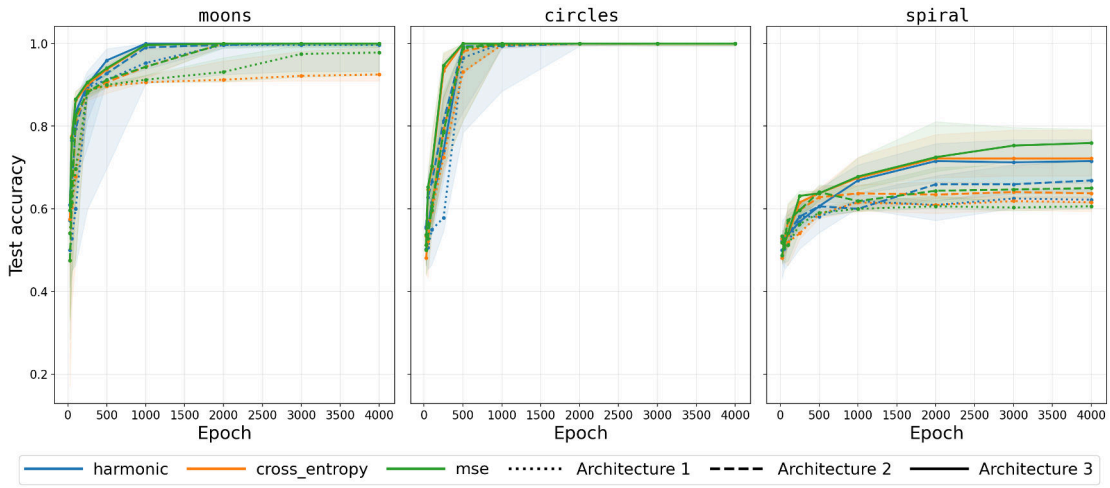


Figure 4.1: Test accuracy over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.1 shows the test accuracy over training. The models learn the classification problem for the `moons` and `circles` datasets quickly under all three loss functions. On `moons`, the test accuracy rises sharply during the first few hundred epochs and then stays high throughout. But CE on A1 does not reach full separation and stays around 0.9 after training, while MSE also takes substantially longer to reach an accuracy of 0.98. On `circles`, most architectures and losses approach nearly perfect test accuracy by the checkpoint around epoch 1000 with a median test accuracy of 1.0. The classification of the `spiral` dataset is clearly harder to solve. The three architectures do not all reach the same performance. Performance increases from A1 to A3 as network depth increases. Loss-dependent differences remain moderate for A1 and A2. On `spiral`, final median accuracy increases from approximately 0.61–0.62 for A1, to 0.64–0.67 for A2, and to 0.72–0.76 for A3, depending on the loss function. These results satisfy the intended experimental design, since we want the classification of `circles` and `moons` to be

solvable, while classification of `spiral` remains only partially solved, so we can analyze the edge cases of not fully solved classification problems.

4.2 Reliability of the Grid Approximation

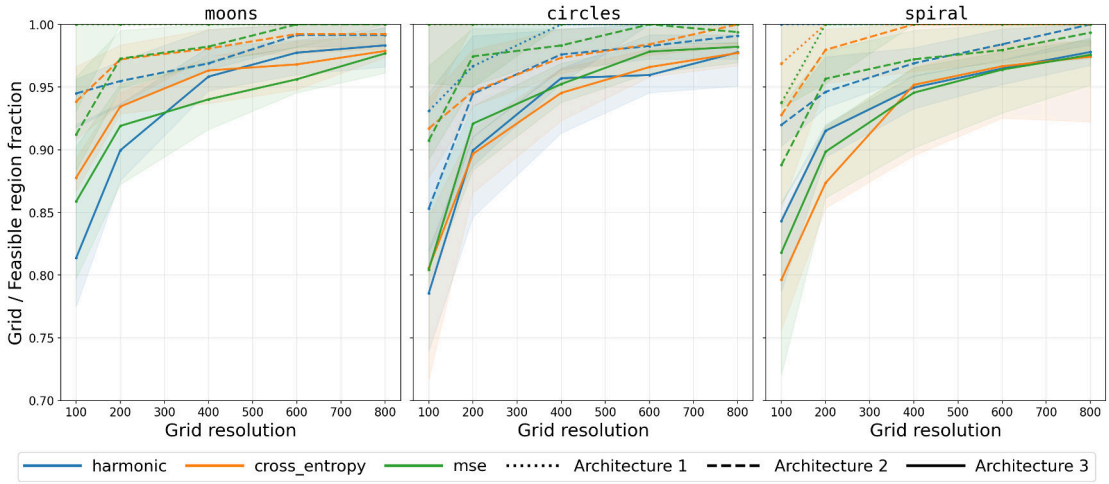


Figure 4.2: Reliability of grid activation-region counts at epoch 4000. The proportion of grid region count to feasible region count is shown for different grid resolutions. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.2 shows the fraction of recovered activation regions by the grid approach at epoch 4000. For each loss function and architecture we have a separate curve, which shows the respective recall. We can observe that A1 is only weakly affected by the grid resolution. At the coarsest resolution, its median recall ranges from approximately 0.93 to 1.00 across datasets and losses. From resolution 400 onward, the median recall is 1.00 in all configurations, so A1 essentially recovers the feasible region count once the grid is moderately fine. A2 shows a more visible resolution effect: at resolution 100, the median recall ranges from approximately 0.85 to 0.93, and it increases steadily with resolution. At resolution 800, the median recall lies between approximately 0.98 and 1.00. A3 is the most resolution-sensitive architecture. At resolution 100, its median recall ranges from approximately 0.75 to 0.87. The lowest values occur mainly on `circles` and `spiral`, where several configurations remain below 0.80 at the coarsest resolution. At resolution 800, A3 reaches median recall values between approximately 0.95 and 0.98.

In conclusion, all networks reach high median recall at resolution 800, with values of at least approximately 0.95. Therefore, the grid approach is sufficiently reliable for the main analysis, but it should still be interpreted as conservative because it can underestimate the feasible activation-region count, especially for A3.

4.2.1 Qualitative Evolution of Activation-Pattern Partitions

In this section we give the reader a visualization of how activation regions can develop over time during training of the model. To illustrate this, we show activation-region partitions for each loss function as an example. This visualization does not replace our quantitative metrics, but we present this as an opportunity for the reader to get a better understanding of what we measure.

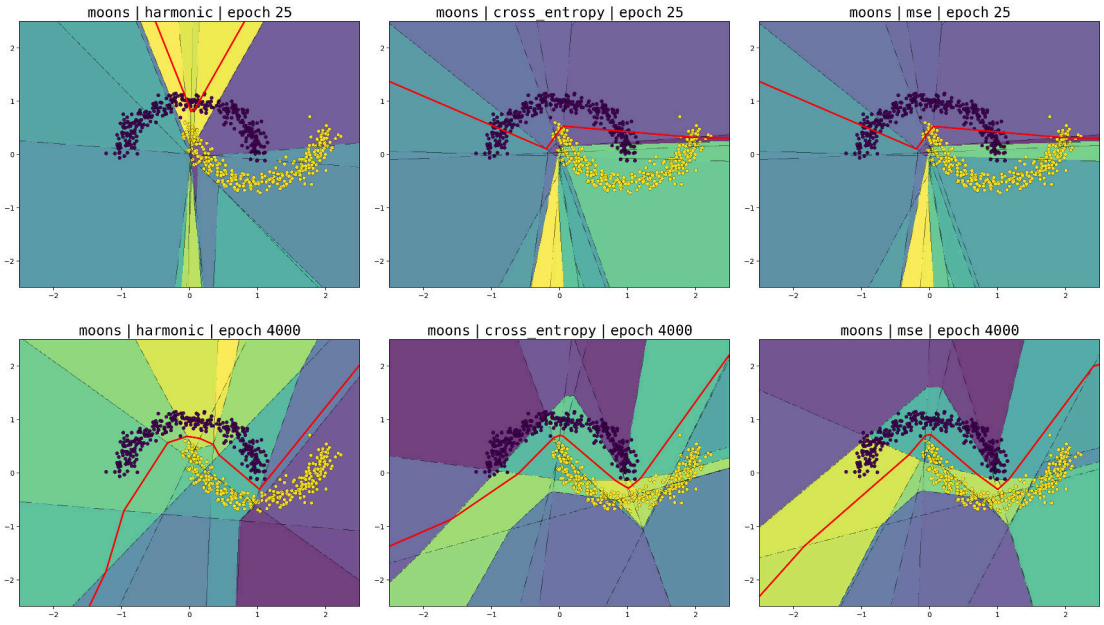
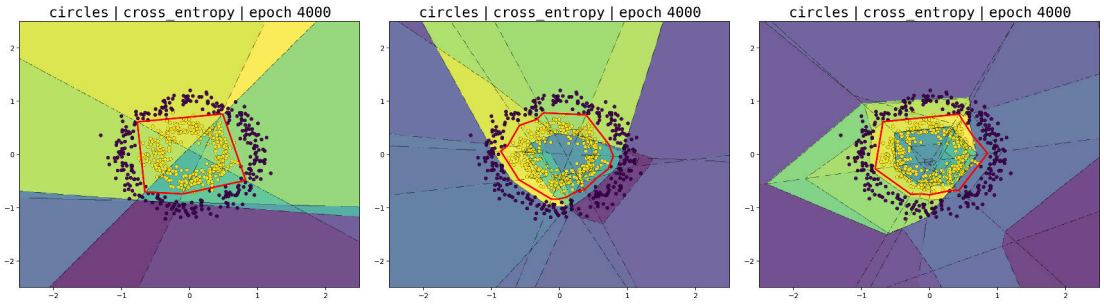


Figure 4.3: Activation region visualizations on the `moons` dataset for A2 with seed 1. The top row shows epoch 25, and the bottom row shows epoch 4000. The colored background shows the activation region partition, while the red curve shows the decision boundary.

Figure 4.3 compares the first checkpoint at epoch 25 with the final checkpoint at epoch 4000 to show the evolution of activation regions over the course of training. At epoch 25, the partition of activation regions appears relatively unstructured and does not follow the data geometry, and the decision boundary is highly irregular. But by epoch 4000, the decision boundary follows the two-moons structure closely for all loss functions and the activation regions appear more aligned with the geometric shape of the data manifold. We also see that HL produces a visibly different partition from CE and MSE even when their final classification accuracy is similar. After illustrating loss-dependent differences on one example, we now visualize how the choice of architecture affects the activation region partition. For that, we chose `circles` and CE.



(a) Architecture 1. **(b)** Architecture 2. **(c)** Architecture 3.

Figure 4.4: Activation region partition and decision boundary on the `circles` dataset for CE loss at epoch 4000. All three architectures are shown for seed 1.

Figure 4.4 shows that the decision boundary is fully closed and separates the inner and outer classes nearly perfectly in all cases. But we note that the deeper the network gets, the finer the partition gets, which is consistent with the activation region count results, where deeper networks have notably more regions. But we can also see that the difference between A2 and A3 is smaller than the jump in regions from A1 to A2.

4.3 Activation-Region Analysis

In this section, we present the main results of our experiments on activation-region count and size, as well as how they interact with the decision boundary. Additionally, we compare the distance between data points and the two types of boundaries: activation boundary and decision boundary.

4.3.1 Region Count and Growth during Training

First, we present the results for the number of activation regions constructed by the different networks and how they evolve over training.

Region Count

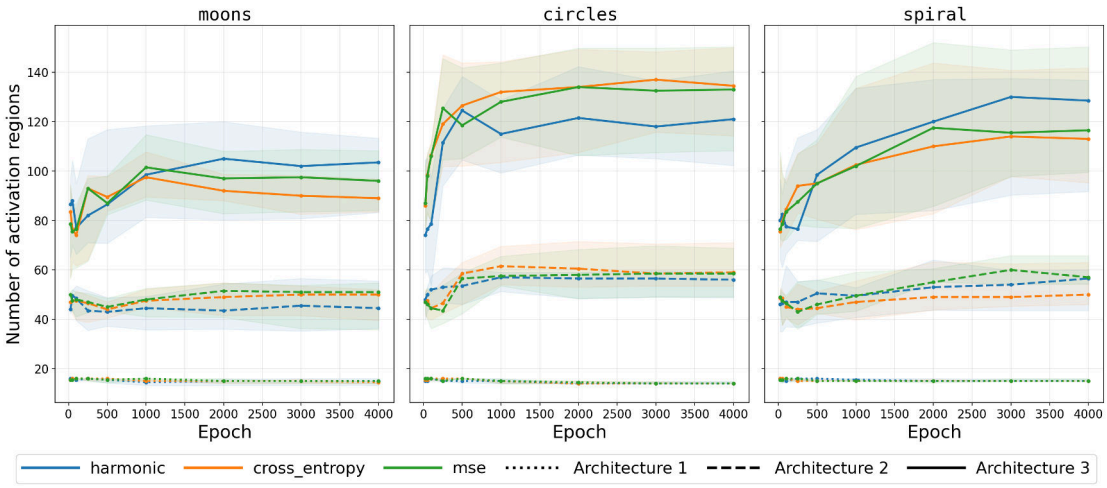


Figure 4.5: Number of activation regions over epochs. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.5 shows the number of activation regions on the grid during training. It is visible that the chosen architecture influences the number of regions, with larger architectures producing more regions and also showing greater growth. A1 constructs about 14 to 15 regions and almost does not change during training across all datasets and losses. A2 creates substantially more regions, between 45 and 59 at the final checkpoint. A3 creates the most regions, around 89 to 104 on `moons`, around 121 to 135 on `circles`, and around 113 to 129 on `spiral`. `circles` and `spiral` both create finer partitions than `moons` on A3, even though the classification problem on `circles` is solved more easily in terms of test accuracy. On `circles`, the growth in activation regions is steep early in training and then largely saturates around epoch 500. On `spiral`, the increase is more gradual but continues for longer, until around epoch 3000, especially for A2

and A3, even though `circles` can reach a similarly high or higher final region count. Within each architecture, the loss-dependent differences are smaller than the differences between architectures. The direction of the loss effect depends on the dataset and architecture: for example, HL produces more regions for A3 on `moons`, but fewer regions than CE and MSE for the deeper architectures on `circles`.

Region Growth

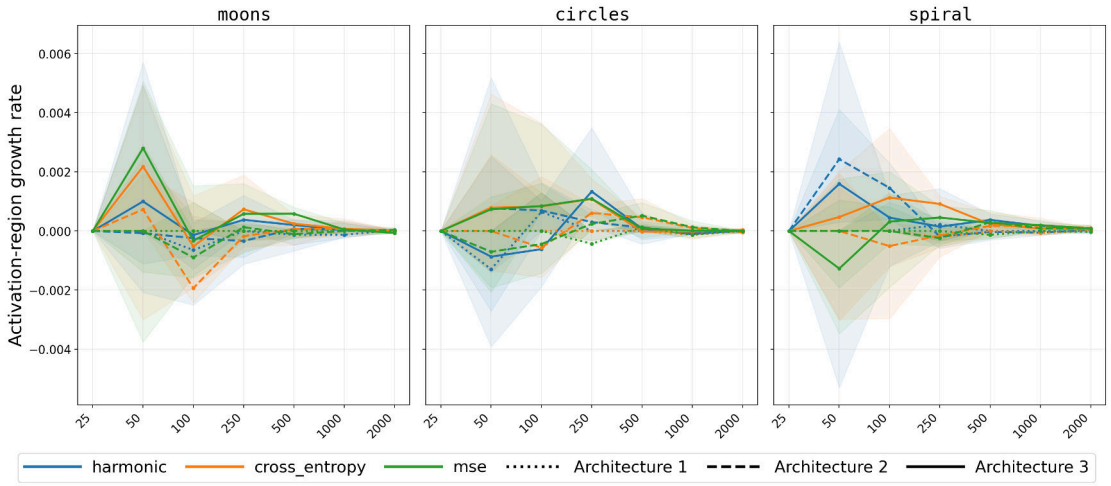


Figure 4.6: Log growth rate of the activation-region count between checkpoints. Checkpoints are shown with equal horizontal spacing to make early training dynamics more visible, and the plot is restricted to epochs before 2000, after which the growth rates are close to zero in most configurations. Curves show the median over 10 random seeds, and shaded regions indicate the 25th–75th percentile range.

In fig. 4.6 we observe that the largest changes in activation-region count occur in the early training phase. The interquartile bands are widest in this phase and include both positive and negative values, which means that the growth rate is highly volatile at the beginning of training. After approximately 500 epochs, most log-growth rates move closer to zero, which indicates that the rapid phase of region creation and removal stabilizes. Nevertheless, the stabilization is not equally strong across datasets. On `moons` and `circles`, the region count largely stabilizes earlier. On `spiral`, especially for A2 and A3, small positive growth persists for longer and accumulates into a continued increase in the absolute number of activation regions as we can see in fig. 4.5.

4.3.2 Area Geometry

After measuring the quantity of regions, we look at the geometric properties of the activation region partition. These help us to understand how much space activation regions occupy and how evenly distributed they are.

Region-Area Fraction

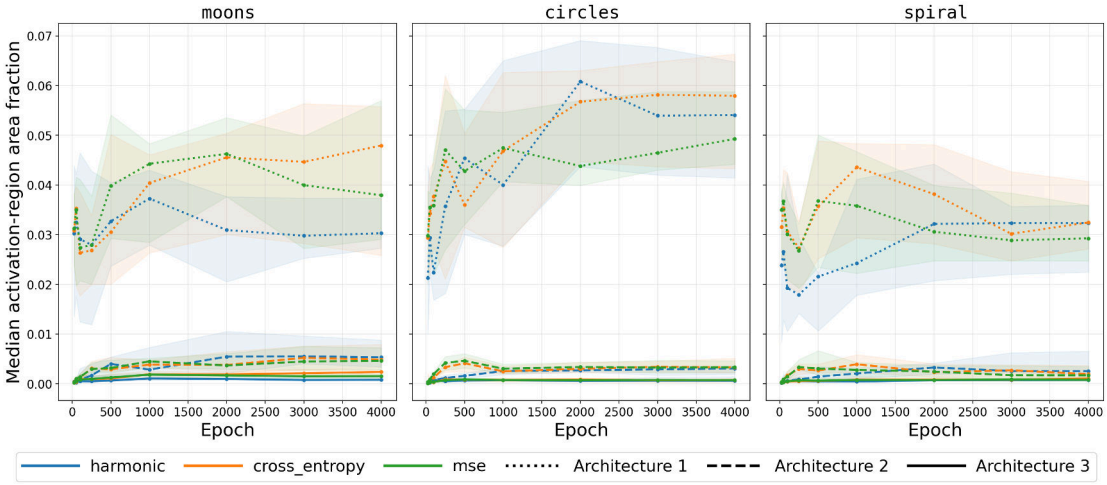


Figure 4.7: Median activation-region area as a fraction of the total bounded input area. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.7 shows that the dominant effect is the network architecture: across all three datasets, A1 has by far the largest median region areas, while A2 and A3 remain close to zero throughout training. On `moons`, A1 reaches 0.03 to 0.048 at the final checkpoint. On `circles`, it reaches 0.049 to 0.058, while on `spiral`, it stabilizes around 0.029 to 0.033. The deeper architectures, A2 and A3, remain much smaller across all datasets, with A2 mostly around 0.003 to 0.009 and A3 mostly around 0.001 to 0.002.

The median area fraction follows the inverse trend of the activation-region count metric. Loss-dependent differences are visible but not consistent across datasets and architectures, and are smaller than the architecture and dataset effects.

Region-Area Gini Coefficient

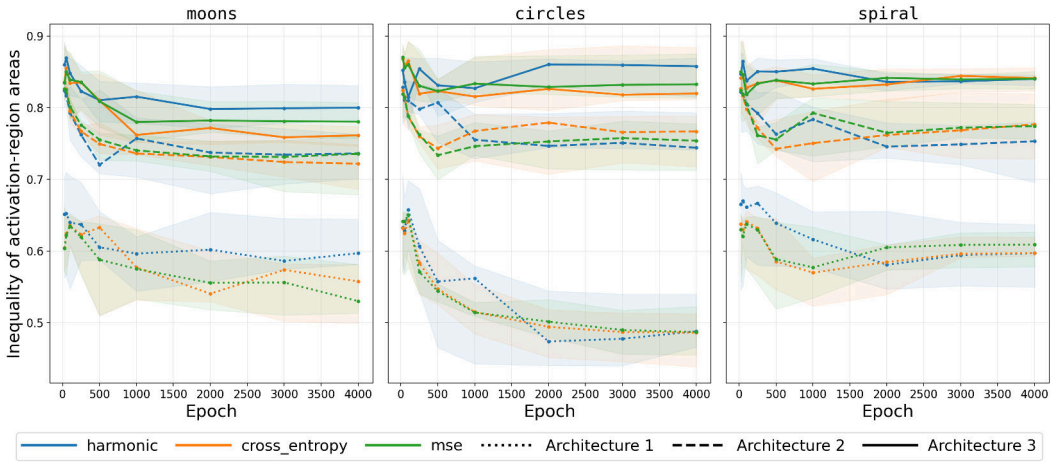


Figure 4.8: Gini coefficient of the activation region area. Higher values signify a more unequal region size distribution. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

While fig. 4.7 measures the median region size, the Gini coefficient in fig. 4.8 measures how unevenly these regions are distributed over input space. On `moons`, A2 remains around 0.72, while A3 ranges from about 0.77 to 0.81 at the last epoch. On `spiral`, A2 reaches about 0.75 to 0.79, and A3 reaches about 0.83 to 0.85. A1 scores lower, around 0.53 to 0.62 depending on the dataset. On `circles`, we see the strongest difference, with A1 dropping to roughly 0.45 to 0.55 and A3 remaining near 0.75 to 0.85. So here A1 builds the most homogeneous partition, which we can see quite clearly in the representative partition in fig. 4.4. Again, the main trend is architecture-dependent: A2 and A3 score the largest Gini coefficients and A1 the lowest, and the increase from A2 to A3 is smaller than from A1 to A2.

4.3.3 Decision-Boundary Interaction

We now know that regions are unequally distributed, but we still do not know where exactly activation regions cluster in the input space. For this, we report how activation regions interact with the decision boundary and whether they concentrate close or far away from the decision boundary.

Boundary Regions

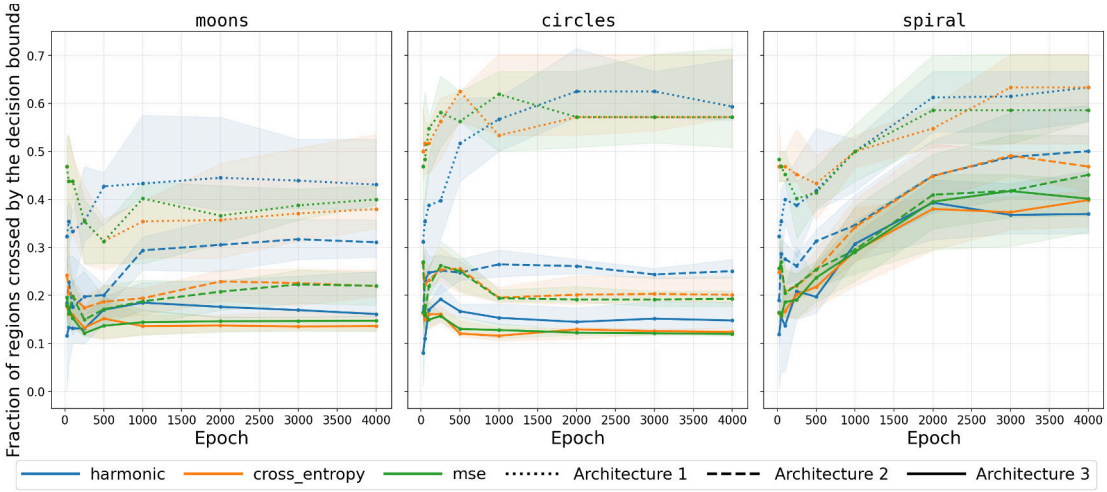


Figure 4.9: Fraction of decision-boundary-touching regions over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.9 shows the fraction of activation regions that directly cross the decision boundary. On `moons`, we see a hierarchy across architectures, with A1 having the highest value and A3 the lowest. This effect develops after the early training phase around epoch 500. Then A1 reaches the highest fraction after training, with HL increasing to about 0.44 and CE and MSE stabilizing around 0.42. A2 remains lower, ranging from about 0.20 for CE and MSE to about 0.33 for HL, while A3 shows the smallest boundary region fraction, with values around 0.14 to 0.19. On `circles`, we see even stronger architecture-dependent differences. The shallow architecture, A1, places the highest share of activation regions on the boundary, around 0.59 to 0.60 for all three loss functions. A2 lies in an intermediate range, around 0.20 to 0.26, while A3 ranges from around 0.12 to 0.17. On `spiral`, the boundary-region fractions start at similar values to `moons` but then rise continuously almost until the final epoch 4000. The curves increase strongly during the first 1000 to 2000 epochs and then stabilize more. A1 reaches a boundary region fraction around 0.59 to 0.63. A2 lies below it, around 0.45 to 0.50, and A3 reaches around 0.37 to 0.40 at the final checkpoint. Compared with `moons` and `circles`, the difference between architectures is smaller. But we note that even though A1 has a high proportion similar to that observed on `moons`, A2 and A3 also reach higher fractions than on `moons` and `circles`, especially A2, which reaches values close to 0.5 for HL. This shows that on `spiral` we see the highest fractions overall after training.

In general, we also see in the early training phase the fraction of boundary regions is growing in all settings, but especially strongly on `spiral` and `circles`. On the hardest dataset, `spiral`,

the fraction of boundary regions grows more slowly but for a longer period than on `circles`. There it grows rapidly but stagnates from epoch 500 on. On `spiral` the boundary-region fraction keeps growing until epoch 2000, which is in line with the accuracy trend in section 4.1. Loss-dependent differences are also visible: Models with HL have the largest boundary-region fraction, especially on `moons` and `circles`, although the differences between losses are mostly small, with the exception of A2 on `moons`. Overall, the architecture and dataset effects are stronger than the loss-dependent differences.

Boundary-Region Area

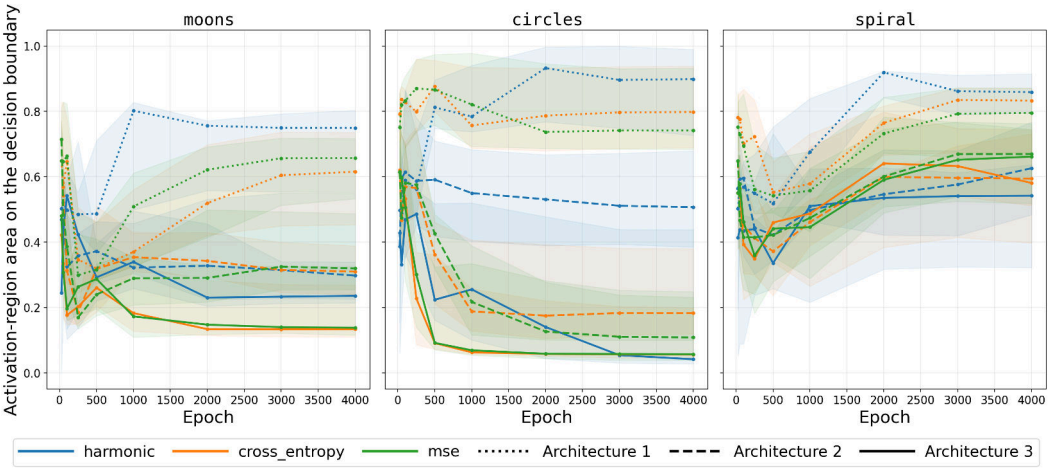


Figure 4.10: Boundary-region area fraction over training. Curves show the median over 10 random seeds; shaded regions show the interquartile band.

Figure 4.10 presents the fraction of area occupied by the boundary regions on the input grid. This is related to the metric above but weights each region by its area. First we note that on all datasets the area fraction on the decision boundary fluctuates strongly during the first 500 epochs. Starting with `moons`, A1 shows the largest fraction with HL reaching the highest value after training, around 0.75, while CE and MSE stabilize around 0.62 to 0.66. For A2, the fraction is lower, around 0.30 to 0.32. The deepest architecture, A3, has the smallest fraction, with values around 0.13 for CE, 0.14 for MSE, and 0.24 for HL. On `circles`, the separation between architectures is similar. A1 again shows the largest fraction, with HL reaching near 0.9 late in training and CE and MSE around 0.75 to 0.8. A2 lies in an intermediate range, with HL having a substantially higher fraction, around 0.5, than CE that stays around 0.18 and MSE around 0.11. For A3 we see much smaller fractions, roughly between 0.04 and 0.06 depending on the loss. We note that the strongest architecture-dependent effect can be seen on `circles`. On `spiral`,

the boundary-region area fraction remains higher overall and fluctuates longer until epoch 1000. A1 mostly reaches values around 0.8 at the final epoch. The deeper architectures show lower fractions here on the boundary than A1, with fractions that remain between roughly 0.55 and 0.75 by the end of training. But this difference between shallow and deep architectures is even stronger on `moons` and `circles`. HL produces in most settings the largest fraction of boundary-region area, while CE and MSE usually produce fractions closer to each other.

Illustration of Boundary Structure for Harmonic Loss

To illustrate the high boundary region fraction observed for A1 with HL on `circles`, fig. 4.11 shows an example activation region partition at epoch 4000. The partition consists of a small number of large regions, and the decision boundary intersects most of them. In this example, the large affine regions form a simple decision-boundary structure. This visual pattern is consistent with the high fraction of boundary regions and their high area fraction reported for A1 on `circles`.

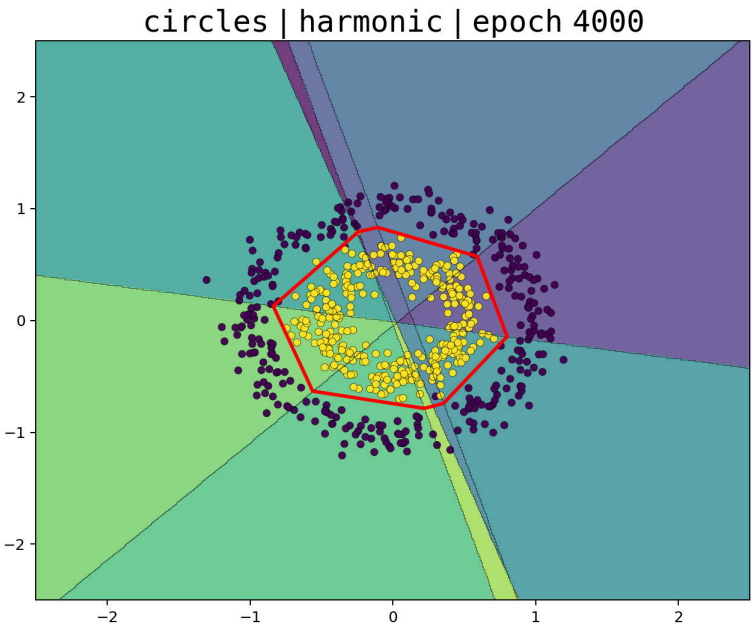


Figure 4.11: Example of activation region partition for the `circles` dataset with HL and A1 at epoch 4000 for seed 4. The colored background shows the activation regions and the red curve shows the decision boundary.

Decision-Boundary Band

This metric is strongly connected to the results of our boundary-region metric, since it uses the fraction of regions exactly on the decision boundary after training is complete. Section 3.3.3 measures how many activation regions lie close to the decision boundary as a band around the boundary is widened by ϵ. Here, ϵ is the half-width of the distance band $[-\epsilon, \epsilon]$ in input space.

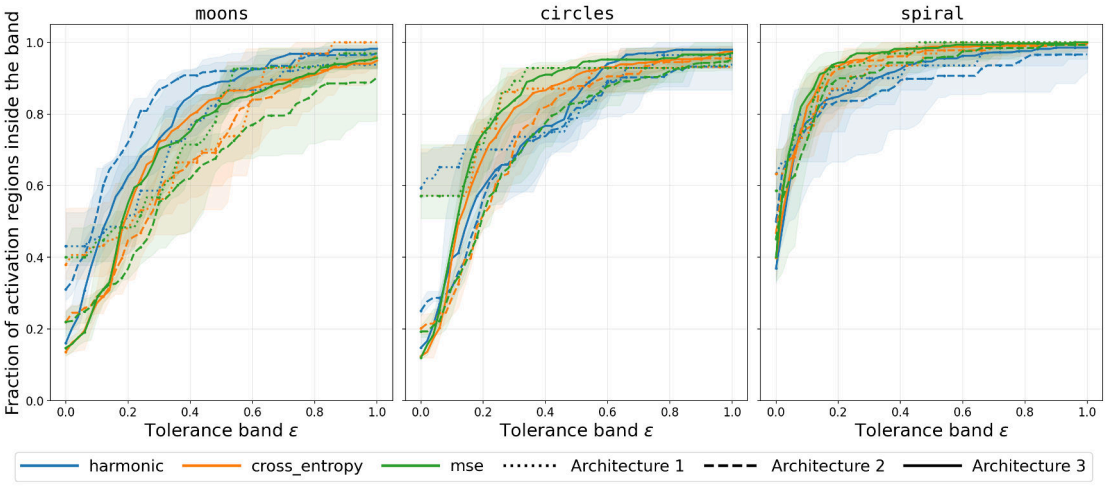


Figure 4.12: Decision band sweep at epoch 4000. The y-axis shows the fraction of activation regions that intersect the $[-\epsilon, \epsilon]$ band. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.12 shows the architecture comparison for $\epsilon \in [0, 1]$. All curves increase rapidly for small values of ϵ and gradually flatten as ϵ increases. On `moons`, the increase is gradual. At $\epsilon = 0$, A1 starts highest, with median fractions around 0.38 to 0.43. A2 starts around 0.22 to 0.31, while A3 starts lower, around 0.14 to 0.16. At $\epsilon = 0.6$, the median fractions on `moons` range from approximately 0.77 to 0.93. On `circles`, A1 already has a high median fraction at $\epsilon = 0$, around 0.57 to 0.59. The deeper architectures start much lower, with A2 around 0.19 to 0.25 and A3 around 0.12 to 0.15, but they increase strongly as the band is widened. On `spiral`, the curves rise the fastest. Already at $\epsilon = 0.2$, the median decision-boundary-band fraction reaches approximately 0.84 to 0.94 across architectures and losses. Overall, the sweep shows that activation regions cluster strongly near the decision boundary, especially on `spiral`. However, `moons` shows a more gradual increase, so here the fractions become larger for wider bands such as $\epsilon = 0.4$ or $\epsilon = 0.8$.

Distance between Data Points and Decision Boundary

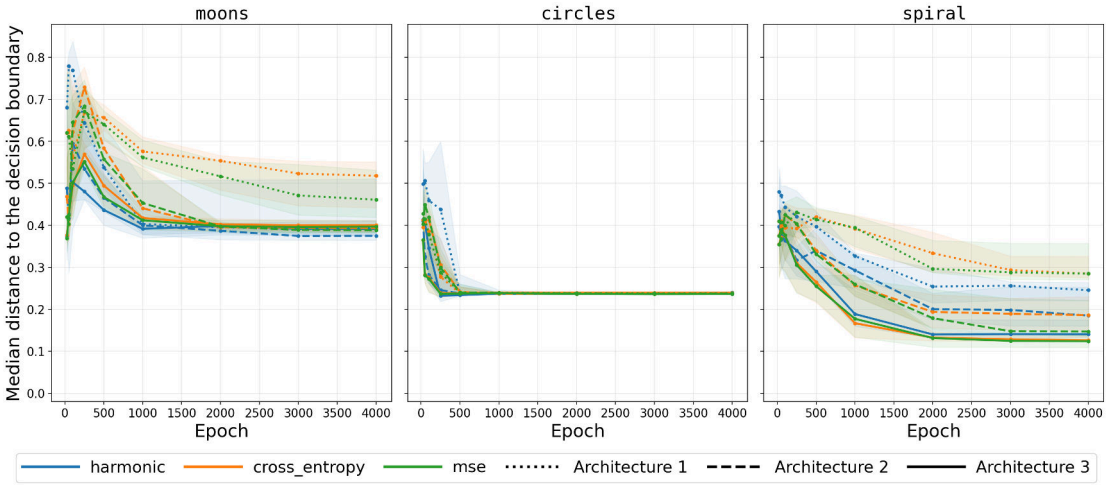


Figure 4.13: Median decision boundary distance to data points over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.13 shows the median distance from the training samples to the local decision boundary over training. On `moons`, the curves show the largest variation during the first 500 epochs. HL has a visible early peak of around 0.8 for A1, but quickly falls like all other curves. After the first few hundred epochs, the trend becomes more stable and after epoch 1000, most distances lie roughly around 0.4. The only exceptions are CE and MSE for A1 which stay at higher distances around 0.5. On `circles`, the distances are lower than on `moons`. After the first 500 epochs, all curves stabilize at approximately 0.25. The differences between loss functions and architectures are marginal since all values stabilize at similar values. On `spiral`, the distances also decrease during the early part of training. The curves start around the range of approximately 0.45 and decrease during training. At the final checkpoint, A1 remains around 0.25 to 0.28, A2 around 0.15 to 0.19, and A3 around 0.12 to 0.14.

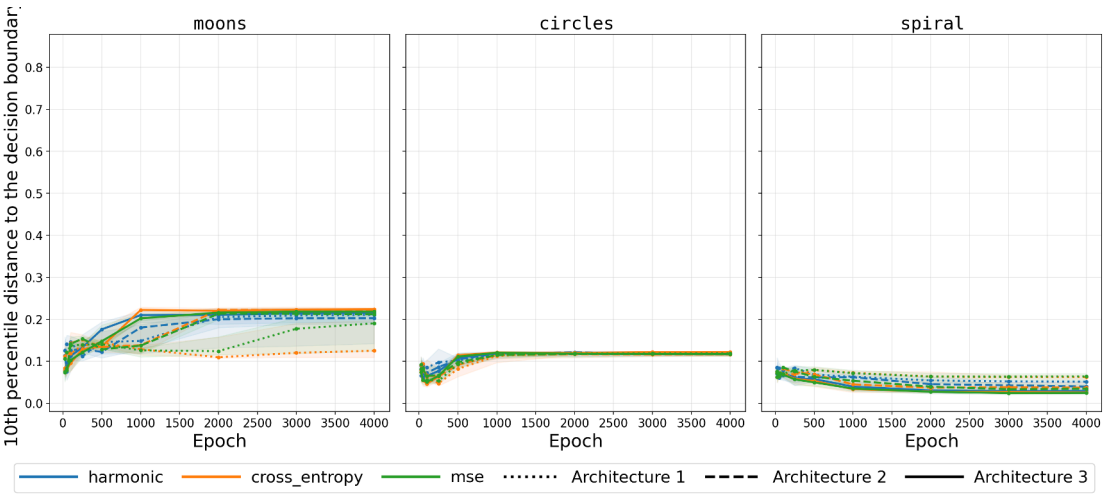


Figure 4.14: Comparison of the 10 percent quantile decision boundary distance over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.14 shows the 10th-percentile distance, corresponding to the 10% of training samples closest to the decision boundary. We see a similar trend of strong stabilization with narrow interquartile bands like in the median distance results. One difference is that in the beginning the curves start at lower distances and then grow into their stable configuration, while in the median distance results, distances start higher and shrink into their stable configuration. On `moons`, most configurations stabilize at approximately 0.20. The main exception is CE with A1, which reaches a lower median distance of approximately 0.13. MSE with A1 also remains slightly lower at approximately 0.19, but HL with A1 reaches approximately 0.21. For A2 and A3, the final values are similar across the three loss functions. On `circles`, all curves stabilize at approximately 0.12. On `spiral`, the distance decreases with architecture depth: it lies at approximately 0.05–0.06 for A1, 0.035–0.040 for A2, and 0.025–0.030 for A3.

Overall, fig. 4.13 and fig. 4.14 show that the median and 10th-percentile decision-boundary distances change mainly during the early training phase and then stabilize for all three datasets. The absolute distance levels differ across datasets: `moons` has the largest later-stage distances, followed by `circles`, while `spiral` has the lowest.

4.3.4 Activation Boundary Interaction

After analyzing the distance to the decision boundary, we also take into account how far data points are from their own activation region boundary. The results are reported separately for the unscaled distance and then scaled by neuron count.

Distance between Data Points and Activation Boundary

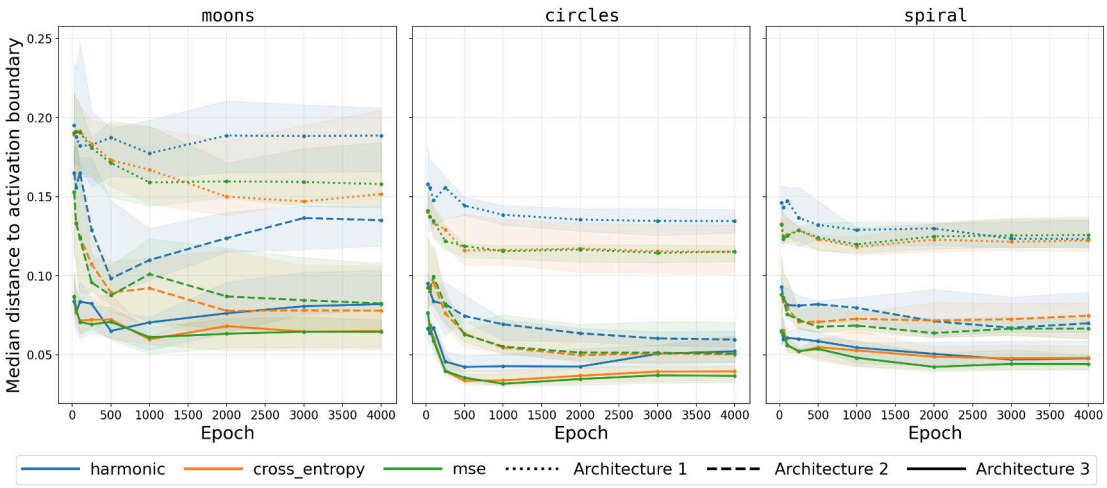


Figure 4.15: Median activation-boundary distance for data points over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.15 shows the median distance from the data points to the activation boundary. On `circles` and `spiral`, all curves decrease during early training and then stabilize. We note that from A1 to A3 the median distance for a data point to the nearest activation boundary shrinks. A1 remains clearly separated from the deeper architectures, with final values around 0.12 to 0.13 on `circles` and `spiral`, and around 0.15 to 0.19 on `moons`. On `moons`, mostly we see the same trends but with a weaker separation than on the other two datasets. Also, we note that HL on A2 increases to a substantially higher value around 0.15, than CE and MSE, which remain lower at 0.07. The depth of the network shows a clear ordering on these datasets. Loss-dependent differences are small compared with the architecture effect. This metric supports the interpretation from the region-count and area-geometry results. Deeper networks form a finer activation partition, and the data points therefore lie closer to hidden activation boundaries. The loss function affects individual curves, but the depth-dependent order is the main effect.

Activation Boundary Distance Scaled to Neuron Count

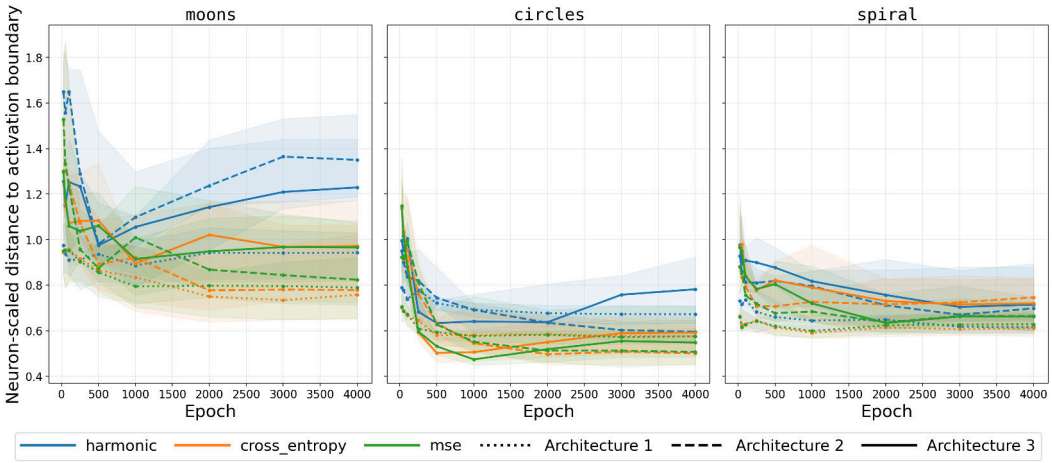


Figure 4.16: Neuron-scaled median activation-boundary distance over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range.

Figure 4.16 shows the median activation-boundary distance multiplied by the number of neurons in each model. This scaling allows the unscaled distances to be compared relative to the number of hidden neurons in each architecture. Overall, most curves exhibit a downward trend over training. In many cases, the largest values occur during the early training phase, followed by a decrease as training progresses. The main exception occurs for HL on `moons` as seen in the unscaled distance metric. While these curves initially decrease similarly to the other loss functions, they increase again after roughly epoch 500, which leads to the largest distances observed in the metric.

5 Discussion

This chapter discusses the main implications of the experiment. The central finding is that predictive accuracy alone does not determine the geometry of the learned activation-region partition. Across the experiments, architecture and dataset geometry have the strongest influence on the number, size, inequality, and boundary involvement of activation regions. The loss function also affects the learned partition, but its influence is more setting-specific and weaker compared to the effects of architecture and dataset geometry. The discussion is divided into the first parts of grid approximation, accuracy and training dynamics. This is followed by a discussion of the main activation-region results on architecture, dataset geometry, and loss function. The chapter ends by distinguishing activation-region stability from classification robustness and by outlining the main limitations.

5.1 Grid Approximation

Before interpreting the main results, it is important to consider the reliability of our grid approximation that we have chosen for our experiments. The resolution analysis in section 4.2 shows that the 800×800 grid used in most experiments from eq. (3.4) captures most feasible activation regions in the bounded input space Ω defined in eq. (3.3). For smaller architectures, the grid almost always recovers the feasible region count at sufficiently high resolution. For deeper architectures, some very small regions may still be missed, since they are harder to resolve on a grid, as illustrated conceptually in fig. 3.4.

These missed regions can matter, because neighboring activation regions can form a small local irregularity of the decision boundary as in fig. 5.1. This can create a small island close to the data manifold and thereby reduce locally the distance between data points and the decision boundary [1]. This can become problematic for classifier behavior if part of the decision boundary is so close to the data that a small perturbation is enough to change the classification output. This is consistent with the boundary-tilting perspective, in which adversarial examples arise when the decision boundary lies close to the data manifold [32]. Therefore, the grid approximation should not be interpreted as an exact reconstruction of the full activation region partition, but it is sufficient for the comparisons made here.

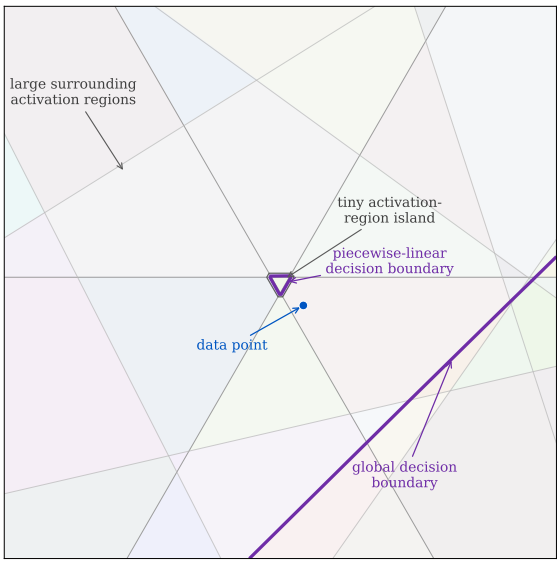


Figure 5.1: Example of activation regions and the decision boundary that form an island close to a data point. *Source: author's own illustration.*

5.2 Accuracy and Geometry

The performance results show that similar classification accuracy does not imply similar activation-region geometry. This is most visible on the `circles` dataset, where most architectures and loss functions reach almost perfect test accuracy by the end of training. However, the corresponding activation-region metrics still differ substantially between architectures: deeper networks realize more activation regions, have smaller median region areas, and show more unequal region-size distributions, for example. This means that predictive performance and activation-region geometry capture different aspects of the learned model. Test accuracy describes whether the classifier assigns the correct labels, whereas the activation-region metrics describe how the model partitions the input space in order to obtain this classification. In consequence, the activation-region analysis reveals geometric differences between models that would remain hidden if only test accuracy were considered.

5.3 Training Dynamics

The activation region growth results in fig. 4.6 show that the strongest changes in region count take place during the first checkpoints, shown by the log growth rate having its largest positive and negative values during the first checkpoints. However, this does not imply that the geometry becomes completely fixed after accuracy reaches its relative peak. Comparing fig. 4.1 with fig. 4.6 shows that accuracy can stabilize while the activation regions still undergo geometric changes. These results are related to the findings of Humayun et al. [33]. They

report a phase transition during training after accuracy values stabilize, during which activation region density moves away from the training samples and clusters near the decision boundary. We also see a growth of activation regions throughout later training epochs, especially on the more complex dataset `spiral`. One possible explanation is that representing the more complex class separation here requires greater partition complexity. On `circles`, region growth stabilizes earlier than on `spiral`, which is consistent with the simpler geometry of the circular decision boundary. Later training then appears to refine an already established partition rather than completely reorganizing it. This interpretation is consistent with previous work relating boundary complexity to finer learned partitions [22].

5.4 Architecture Effects

The region-count results in section 4.3.1 show a consistent architecture ordering: A1 creates the fewest regions, A2 creates more, and A3 creates the most. This ordering appears across datasets and losses.

5.4.1 Region Capacity

Increasing depth raises the number of possible activation patterns (Equation (3.8)). This combinatorial count can be compared with the tighter upper bound on realizable regions given by Raghu et al. [34]. They define the upper bound on possible regions for a network with L hidden layers of width k and input dimension d as

$$N_{\text{regions}} \leq \left(\sum_{j=0}^{d} \binom{k}{j} \right)^{L}.$$

For $d = 2$ and $k = 5$, this gives upper bounds of 16, 256 and 4096 regions for A1, A2 and A3. We can see that the numbers of realized activation regions in our experiment are still much smaller: A2 usually realizes around 45 to 59 regions, while A3 reaches roughly 90 to 135 regions depending on the dataset. The networks use only a small subset of their combinatorial activation region capacity, which is consistent with Hanin et al. [9], who showed that models usually realize far fewer activation regions than the theoretical upper bound.

5.4.2 Region Size and Inequality

At the same time, the area-geometry results in figs. 4.7 and 4.8 show that the median region area decreases with depth, while the Gini coefficient increases. This indicates a more unequal size distribution, with many small regions and a smaller number of large regions. In the representative A3 visualization in fig. 4.4, many of the small regions appear near the data and decision boundary, although this pattern cannot be generalized from the example.

5.4.3 Boundary Involvement

Architecture depth also affects how activation regions interact with the decision boundary. The results for the boundary-region metric in fig. 4.9 and the boundary-region area metric in fig. 4.10 show that A1 has the largest direct boundary involvement. One possible explanation is that A1 has fewer and larger regions, which can increase the probability that regions lie on the decision boundary. For deeper architectures, fewer regions are directly crossed by the boundary. However, the decision-band analysis in fig. 4.12 shows that this difference becomes smaller once the neighborhood around the decision boundary is slightly widened. Even for small values of ϵ, all architectures follow a similar trend and place a large fraction of their activation regions in the close environment of the boundary. When the band is widened further, the deeper architectures increasingly approach the fraction of near-boundary regions observed for A1. Overall, all architectures follow the same trend in having a high near-boundary fraction throughout the sweep, but the deeper networks keep smaller and more localized regions than the shallow network A1. This again points to additional depth introducing localized geometric detail near the boundary (cf. section 5.3).

5.5 Dataset Effects

The performance results in section 4.1 show that `moons` and `circles` are largely solved by most models, while `spiral` remains the hardest dataset to solve for the classifiers here. Therefore, we now discuss how activation region partitions behave differently depending on the complexity of the classification problem.

5.5.1 Boundary Concentration

On easier datasets, additional activation regions can be created without all of them being needed for class separation. However, the amount of boundary concentration is not determined only by classification difficulty. It is also affected by the geometric form of the decision boundary that has to be represented. For example, `circles` is solved more easily than `spiral`, but a closed circular decision boundary still requires several small activation regions to approximate the full circular separation. In contrast, `moons` can be separated with fewer boundary segments in this experimental setting, even though in the accuracy plots we can see that it takes more epochs to solve. The decision-boundary band results from section 4.3.3 strengthen this interpretation by showing a clear ordering between the three datasets: from `moons` to `circles` to `spiral`, the models place progressively larger fractions of their activation regions near the decision boundary. Even on `moons`, the easiest dataset, a large fraction of regions already lies near the decision boundary once the band is modestly widened. The effect is even more drastic on `spiral`, where the boundary regions and boundary-region area are higher and the fraction on the

decision-boundary band rises even faster. These results suggest that the harder dataset requires a larger share of the activation-region partition near the decision boundary to approximate the more complex separation. At the same time, the comparison with `circles` shows that boundary concentration is not only linked to dataset difficulty. It can also reflect the number of activation regions needed to accurately represent the shape of the class separation. A closely related observation is reported by Bosca et al. [35] for the `circles` dataset. They visualize the activation region partition of a trained ReLU network and observe a concentration of activation regions along the decision boundary.

Overall, these results may indicate a saturation effect in the fraction of activation regions on the decision boundary. Once a task is solved with a sufficient boundary, as in `circles` in contrast to `spiral`, a possible interpretation might be that deeper networks do not need to place more regions at the boundary. This might either lead to a deeper network generating fewer regions in general, as we see on `moons`, or to some of the additional regions appearing further away from the decision boundary. When the task is more geometrically complex, as in `spiral`, a larger fraction of the activation partition seems to be needed to refine the decision boundary and its surrounding environment.

5.6 Loss Effects

Throughout our experiments, the influence of loss functions on the partitioning of activation regions is small, except in some settings: HL involves slightly more regions on the decision boundary, especially for the smaller architectures A1 and A2, while CE and MSE often have similar results in figs. 4.9 and 4.10. Stronger differences were seen for A1 on `circles`, discussed in section 4.3.3, where HL produces large activation regions that are almost all on the decision boundary. Also, for A2 on `circles`, we see a substantially higher boundary-region-area fraction than with other loss functions. A possible interpretation for this exception might be that, in smaller networks, the prototype-based structure of HL leads to a more centralized separation of the classes in embedding space, as described in section 2.3.2. On a dataset such as `circles`, this could lead to a circular input-space partition where all regions are crossed by the circular decision boundary. The resulting partition is therefore directly involved in the class separation. In deeper architectures, the additional hidden layers provide enough capacity to create many small regions, and the loss-dependent differences become less dominant than the architecture-dependent ones. Another observed effect of HL appears in the activation-boundary distance metric on `moons` and `circles`. In contrast to the general downward trend, the distance for HL increases steeply again after the early training phase on `moons` and on `circles` we also see that HL consistently produces higher distances for all architectures. A possible interpretation is that, once the decision boundary is already sufficiently learned, HL continues to reorganize the

activation regions so that samples lie deeper inside their activation regions. One hypothesis is that HL indirectly affects the placement of activation boundaries around the samples. Baek et al. [3] establish the class-center interpretation of HL, but they do not study activation-region distances, so this remains speculative. For `moons`, where the data manifolds are curved but still well separable, this can lead to larger distance to activation boundary around the data. As discussed in section 3.3.4, this means increased activation-region stability rather than improved classification robustness. Overall, the use of HL as a loss function produces differences in boundary region geometry and distance to activation boundary results in some settings.

5.7 Stability and Robustness

5.7.1 Activation-Boundary Distance

The activation-boundary results in section 4.3.4 show that deeper networks place activation boundaries closer to data points. The smaller activation-boundary distances are consistent with the larger number and smaller typical size of regions in the deeper models. The neuron-scaled distance explains this in more detail. Since deeper architectures contain more hidden ReLU neurons, they also contain more potential activation boundaries. After scaling the median distance by the number of hidden neurons, the architecture differences become much smaller. This could mean that the decrease in the unscaled distance metric for deeper networks is largely explained by their larger number of activation boundaries. However, the late increase in the neuron-scaled activation-boundary distance for HL on `moons`, as already discussed in section 5.6, is an exception to the general trend. Therefore, when two models achieve similar accuracy, the model with larger activation-boundary distances can be regarded as more locally stable with respect to its activation region. This is desirable from the perspective of local interpretability, since the same affine function remains valid in a larger area around the data [36]. Related work also treats large local activation regions as a useful geometric property [8]. Crossing a hidden activation boundary changes the activation region, but it does not necessarily change the predicted class. However, this does not imply better classification robustness per se, because robustness also depends on the distance to the decision boundary [8], which is discussed below.

5.7.2 Decision-Boundary Distance

The results in figs. 4.13 and 4.14 show that the choice of dataset has the strongest influence on the distance to the decision boundary. The 10th-percentile results are important because they focus on the samples closest to the decision boundary, which are the most sensitive to small perturbations [2, 8]. Interestingly, our results show that all models converge to a similar dataset-dependent distance without big variations in the interquartile band, as we can see in fig. 4.14, where models start with lower distances to the decision boundary but then grow in

distance. At the same time, we see in fig. 4.13 that for the median data point (which is further away from the boundary) the distance to the decision boundary shrinks over training to a stable value. This shows that the distances at the lower end of the distribution increase relative to the decision boundary, even while the median distance decreases. Since we see nearly all models converging to the same distance values that depend on `moons`, `circles` and `spiral`, we could not identify clear or consistent architecture- or loss-dependent differences. For CE, the increase in distance is qualitatively consistent with margin-increasing behavior discussed in the implicit-bias literature [37]. However, the theoretical results in that literature rely on assumptions that do not directly match our experimental setting. Also, the same explanation is not directly transferable to MSE or HL. Their similar behavior in our experiment should therefore be seen as an observation that needs further investigation to determine whether all three losses follow the same underlying mechanism.

5.8 Limitations

There are several limitations to this thesis. In the first place, the experiments are performed on two-dimensional datasets. In higher dimensions, the geometry of activation regions may behave differently and whether our results also hold in higher dimensions needs to be validated. Also, all activation-region metrics are restricted to our bounded input space and therefore describe the geometry inside the chosen input box. Another aspect is that the main activation-region analysis is based on a finite grid. As mentioned in section 5.1, small regions may be missed by this approach, so our grid approximation does not fully replace the complete feasibility construction. This can affect all metrics that depend on the grid approximation, but the risk is mostly caused by very small regions near the data manifold or near the decision boundary. Apart from that, the experiments are performed on a fixed train-test split. This means that parts of the observations in this thesis could depend on the specific data split. A stronger experimental design could repeat the full analysis over k-fold cross-validation. This would reduce dependence on a single split decision and make the analysis more robust. Then, the architectural comparison is limited to three relatively small networks. With only three architecture choices, the results can indicate trends, but they cannot fully describe the effects of width and depth. Future work should therefore include a larger architecture selection, for example by varying depth and width independently. This would make it possible to generalize the depth and width effects better. Lastly, the connection between activation-region stability and classification robustness is not tested directly in this thesis. We relate our interpretation indirectly to the robustness perspective of Croce et al. [8] by using similar metrics and referring to their results. However, a stronger validation would require explicit robustness evaluations in our own experimental setting, such as adversarial attack tests (e.g., FGSM, PGD, or other attacks) to measure directly

whether models with higher data-to-boundary distances also show higher robustness against adversarial attacks. This could provide a clearer correlation between the activation-region metrics used here and actual adversarial robustness.

6 Conclusion

This thesis investigated how the choice of loss function influences the activation-region geometry of ReLU multilayer perceptrons in different experimental settings. Therefore, we evaluated models trained with HL, CE, and MSE not only in terms of predictive accuracy, but also with respect to their activation-region partitions, to determine whether the loss functions lead to geometric differences in this domain. To study this question, we trained fully connected ReLU networks on the datasets `moons`, `circles`, and `spiral`, using three architectures of increasing depth. These datasets were chosen because they can be visualized directly while still requiring non-linear decision boundaries. We chose to vary these factors to isolate the influence of the loss function in our experimental setup.

First, we evaluated the predictive performance of the trained models. The classification task on `moons` and `circles` was solved well by most architectures and loss functions, whereas `spiral` remained substantially harder to solve for all models. At the same time, accuracy alone was not sufficient since, especially on `moons` and `circles`, different models achieved similar accuracy while forming different activation region partitions. Therefore, we analyzed the learned geometry using several activation-region metrics: region counts and their growth rate, region-area distributions, interactions between regions and the decision boundary, and distances from data points to activation boundaries and decision boundaries. To support the reliability of these measurements, we evaluated the grid approach by comparing grid activation-region counts with feasible activation-region counts. The chosen grid resolution recovered most feasible regions, especially for A1, the smallest network. For the deeper networks, some very small regions may still be missed by the approximation.

All in all, activation-region geometry was shaped more strongly by architecture depth and dataset geometry than by the loss function alone. Increasing depth increased the number of activation regions, but the additional regions were not distributed uniformly: Deeper architectures produced smaller regions on average and more unequal region-size distributions. The choice of dataset also had a strong influence on how this geometric capacity was used. For the more difficult classification problem on the `spiral` dataset, a larger part of the activation-region structure was located near the decision boundary than on the other datasets. This suggests that geometrically harder tasks require a larger share of the activation-region partition near the decision boundary, while easier tasks can often be solved with less fragmentation in the partition. The influence of the loss function choice was rather small and setting-specific in comparison to the effect of architecture and dataset geometry. In the decision-boundary fraction and area results, HL showed higher values than its counterparts in some settings, especially

for A2 on `circles`, where large activation regions were directly on the decision boundary. A possible interpretation might be that HL produces a less fragmented partition around the decision boundary in these specific settings.

The activation-boundary distance results further suggest that smaller networks could be preferable when they have a simpler activation region geometry and already solve the task equally well. In many settings, A1 placed activation boundaries farther away from data points which corresponds to a locally simpler activation region structure around the data manifold. This might be mostly caused by the A1 producing fewer activation regions overall. HL posed an exception here, while having significantly higher activation boundary distances for A2 on `moons`, this suggests that in specific conditions HL can lead to denser representations. The distance to decision boundary results showed that all architectures and loss functions led to the same stable distance values for each dataset.

Overall, the results do not support the conclusion that one loss function generally produces more regions, stronger boundary interaction, or more stable activation-region structures. So the higher activation boundary distance for HL in some settings should not be interpreted as a guarantee for classification robustness, since crossing an activation boundary changes the local affine representation but not necessarily the predicted class.

All of our conclusions should be interpreted within the limits of the experimental setup, which focused on small networks, two-dimensional datasets, and grid-based measurements on a bounded input space. Future work could extend this analysis to wider and deeper networks, higher-dimensional datasets, and additional training settings, such as cross validation for the train/test split. Since dense grid sampling becomes computationally costly and exact feasibility calculation nearly impossible in higher dimensions, such studies would require alternative methods for estimating activation-region geometry. Finally, future research could investigate whether the activation-region quantities studied here are connected to generalization, interpretability, or adversarial robustness.

List of Figures

2.1 Schematic activation regions in a two-dimensional input space. Each colored polygon represents one activation region. The black edges indicate activation boundaries, across which at least one neuron changes its activation state. *Source: author's own illustration.* 4

2.2 Stepwise construction of activation regions from two preactivation functions. Source: author's own illustration, adapted from Pukowski et al. [16, Fig. 1]. . . 7

2.3 Illustration of activation regions in a simple two-dimensional ReLU network, with two hidden layers. *Source: author's own illustration.* 8

2.4 Schematic distinction between activation regions and linear regions. The activation regions A_1 and A_2 have the same affine function and therefore form the single linear region L_1. Source: author's own illustration. 9

2.5 Distances to activation and decision boundaries in a ReLU network. The blue line is distance to the activation boundary and the orange line to the decision boundary. *Source: author's own illustration, adapted from Croce et al. [8, Fig. 1].* 12

2.6 Illustration of first-layer activation regions and the zero set of a second-layer ReLU preactivation. *Source: author's own illustration, based on Lim et al. [23, Sec. 3.2 and Fig. 2].* 16

2.7 Illustration of the activation boundaries of two second-layer ReLU neurons in the left panel and the boundary of one chosen activation region in the right panel. The black outline is its boundary ∂R_s. *Source: author's own illustration.* 17

3.1 Visualization of the three two-dimensional binary classification datasets used in the experiments. 19

3.2 Visualization of the three architecture variants. Source: author's own illustration. 20

3.3 Grid-based construction of sampled activation regions from activation-pattern IDs. 22

3.4 A small feasible activation region can be missed by a coarse grid. 23

3.5 Schematic illustration of the band around the decision boundary. The band grows as the distance tolerance ϵ increases. 29

4.1 Test accuracy over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 32

4.2 Reliability of grid activation-region counts at epoch 4000. The proportion of grid region count to feasible region count is shown for different grid resolutions. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 33

4.3 Activation region visualizations on the `moons` dataset for A2 with seed 1. The top row shows epoch 25, and the bottom row shows epoch 4000. The colored background shows the activation region partition, while the red curve shows the decision boundary. 34

4.4 Activation region partition and decision boundary on the `circles` dataset for CE loss at epoch 4000. All three architectures are shown for seed 1. 35

4.5 Number of activation regions over epochs. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 36

4.6 Log growth rate of the activation-region count between checkpoints. Checkpoints are shown with equal horizontal spacing to make early training dynamics more visible, and the plot is restricted to epochs before 2000, after which the growth rates are close to zero in most configurations. Curves show the median over 10 random seeds, and shaded regions indicate the 25th–75th percentile range. 37

4.7 Median activation-region area as a fraction of the total bounded input area. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 38

4.8 Gini coefficient of the activation region area. Higher values signify a more unequal region size distribution. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 39

4.9 Fraction of decision-boundary-touching regions over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 40

4.10 Boundary-region area fraction over training. Curves show the median over 10 random seeds; shaded regions show the interquartile band. 41

4.11 Example of activation region partition for the `circles` dataset with HL and A1 at epoch 4000 for seed 4. The colored background shows the activation regions and the red curve shows the decision boundary. 42

4.12 Decision band sweep at epoch 4000. The y-axis shows the fraction of activation regions that intersect the $[-\epsilon, \epsilon]$ band. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 43

4.13 Median decision boundary distance to data points over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 44
4.14 Comparison of the 10 percent quantile decision boundary distance over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 45
4.15 Median activation-boundary distance for data points over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 46
4.16 Neuron-scaled median activation-boundary distance over training. Curves show the median over 10 random seeds; shaded regions show the 25th–75th percentile range. 47

5.1 Example of activation regions and the decision boundary that form an island close to a data point. *Source: author's own illustration.* 49

List of Tables

3.1 Overview of experimental factors. 18

Bibliography

[1] C. Szegedy et al. “Intriguing properties of neural networks.” arXiv: 1312.6199 [cs.CV], Accessed: Jul. 1, 2026. [Online]. Available: http://arxiv.org/abs/1312.6199 pre-published (cit. on pp. 1, 48).

[2] I. Goodfellow, A. Courville, and Y. Bengio, *Deep Learning* (Adaptive Computation and Machine Learning). Cambridge, Massachusetts: The MIT Press, 2016, 1 p., ISBN: 978-0-262-03561-3 978-0-262-33737-3 (cit. on pp. 1, 3, 10, 11, 13, 53).

[3] D. D. Baek, Z. Liu, R. Tyagi, and M. Tegmark. “Harmonic Loss Trains Interpretable AI Models.” arXiv: 2502.01628 [cs], Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2502.01628 pre-published (cit. on pp. 1, 10–13, 53).

[4] A. Mao, M. Mohri, and Y. Zhong. “Cross-Entropy Loss Functions: Theoretical Analysis and Applications.” arXiv: 2304.07288 [cs.LG], Accessed: Jul. 10, 2026. [Online]. Available: http://arxiv.org/abs/2304.07288 pre-published (cit. on p. 1).

[5] L. Ciampiconi, A. Elwood, M. Leonardi, A. Mohamed, and A. Rozza. “A survey and taxonomy of loss functions in machine learning.” arXiv: 2301.05579 [cs.LG], Accessed: Jul. 10, 2026. [Online]. Available: http://arxiv.org/abs/2301.05579 pre-published (cit. on pp. 1, 11).

[6] R. Pascanu, G. Montufar, and Y. Bengio. “On the number of response regions of deep feed forward networks with piece-wise linear activations.” arXiv: 1312.6098 [cs], Accessed: Mar. 24, 2026. [Online]. Available: http://arxiv.org/abs/1312.6098 pre-published (cit. on pp. 1, 3, 5, 14).

[7] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio. “On the Number of Linear Regions of Deep Neural Networks.” arXiv: 1402.1869 [stat], Accessed: Mar. 24, 2026. [Online]. Available: http://arxiv.org/abs/1402.1869 pre-published (cit. on pp. 1, 3, 14).

[8] F. Croce, M. Andriushchenko, and M. Hein. “Provable Robustness of ReLU networks via Maximization of Linear Regions.” arXiv: 1810.07481 [cs.LG], Accessed: Jun. 11, 2026. [Online]. Available: http://arxiv.org/abs/1810.07481 pre-published (cit. on pp. 2, 12, 14, 15, 27, 30, 53, 54).

[9] B. Hanin and D. Rolnick. “Deep ReLU Networks Have Surprisingly Few Activation Patterns.” arXiv: 1906.00904 [stat.ML], Accessed: Jun. 7, 2026. [Online]. Available: http://arxiv.org/abs/1906.00904 pre-published (cit. on pp. 4, 9, 50).

[10] T. Serra, C. Tjandraatmadja, and S. Ramalingam. "Bounding and Counting Linear Regions of Deep Neural Networks." arXiv: 1711.02114 [cs], Accessed: Mar. 24, 2026. [Online]. Available: http://arxiv.org/abs/1711.02114 pre-published (cit. on pp. 4–6, 9, 14).

[11] M. Stargalla, C. Hertrich, and D. Reichman. "The Computational Complexity of Counting Linear Regions in ReLU Neural Networks." arXiv: 2505.16716 [cs.CC], Accessed: May 26, 2026. [Online]. Available: http://arxiv.org/abs/2505.16716 pre-published (cit. on pp. 4, 24).

[12] J. Matoušek and B. Gärtner, *Understanding and Using Linear Programming* (Universitext). Berlin ; New York: Springer, 2007, 222 pp., ISBN: 978-3-540-30697-9 978-3-540-30717-4 (cit. on p. 5).

[13] R. Balestriero and R. Baraniuk. "Mad Max: Affine Spline Insights into Deep Learning." arXiv: 1805.06576 [stat.ML], Accessed: Jun. 7, 2026. [Online]. Available: http://arxiv.org/abs/1805.06576 pre-published (cit. on p. 5).

[14] S. P. Boyd and L. Vandenberghe, *Convex Optimization*, Version 29. Cambridge New York Melbourne New Delhi Singapore: Cambridge University Press, 2023, 716 pp., ISBN: 978-0-521-83378-3 (cit. on pp. 5, 24).

[15] Y. Wang, "Estimation and Comparison of Linear Regions for ReLU Networks," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, Vienna, Austria: International Joint Conferences on Artificial Intelligence Organization, Jul. 2022, pp. 3544–3550, ISBN: 978-1-956792-00-3. DOI: 10.24963/ijcai.2022/492 Accessed: May 24, 2026. [Online]. Available: https://www.ijcai.org/proceedings/2022/492 (cit. on pp. 6, 9, 23).

[16] P. Pukowski and H. Lu. "SkelEx and BoundEx: Natural Visualization of ReLU Neural Networks." arXiv: 2305.05562 [cs], Accessed: Mar. 24, 2026. [Online]. Available: http://arxiv.org/abs/2305.05562 pre-published (cit. on p. 7).

[17] A. Berzins, "Polyhedral Complex Extraction from ReLU Networks using Edge Subdivision," 2023 (cit. on pp. 9, 15, 17).

[18] P. Piwek, A. Klukowski, and T. Hu. "Exact Count of Boundary Pieces of ReLU Classifiers: Towards the Proper Complexity Measure for Classification." arXiv: 2306.08805 [stat.ML], Accessed: Jun. 26, 2026. [Online]. Available: http://arxiv.org/abs/2306.08805 pre-published (cit. on pp. 9, 13).

[19] S. Kornblith, T. Chen, H. Lee, and M. Norouzi. "Why Do Better Loss Functions Lead to Less Transferable Features?" arXiv: 2010.16402 [cs.CV], Accessed: May 20, 2026. [Online]. Available: http://arxiv.org/abs/2010.16402 pre-published (cit. on p. 10).

[20] C. M. Bishop, *Pattern Recognition and Machine Learning* (Information Science and Statistics), Softcover reprint of the original 1st edition 2006 (corrected at 8th printing 2009). New York, NY: Springer New York, 2016, 738 pp., ISBN: 978-1-4939-3843-8 (cit. on pp. 12, 13).

[21] R. Balestriero, R. Cosentino, B. Aazhang, and R. Baraniuk. "The Geometry of Deep Networks: Power Diagram Subdivision." arXiv: 1905.08443 [cs], Accessed: Mar. 24, 2026. [Online]. Available: http://arxiv.org/abs/1905.08443 pre-published (cit. on p. 13).

[22] A. I. Humayun, R. Balestriero, G. Balakrishnan, and R. Baraniuk, "SplineCam: Exact Visualization and Characterization of Deep Network Geometry and Decision Boundaries," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Vancouver, BC, Canada: IEEE, Jun. 2023, pp. 3789–3798, ISBN: 979-8-3503-0129-8. DOI: 10.1109/CVPR52729.2023.00369 Accessed: Mar. 24, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/10203818/ (cit. on pp. 13, 15, 17, 50).

[23] C. H. Lim, R. Urtasun, and E. Yumer. "Hierarchical Verification for Adversarial Robustness." arXiv: 2007.11826 [cs.LG], Accessed: Jul. 11, 2026. [Online]. Available: http://arxiv.org/abs/2007.11826 pre-published (cit. on p. 16).

[24] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python." arXiv: 1201.0490 [cs.LG], Accessed: Jun. 20, 2026. [Online]. Available: http://arxiv.org/abs/1201.0490 pre-published (cit. on p. 19).

[25] K. J. Lang and M. J. Witbrock, "Learning to tell two spirals apart," in *Proceedings of the 1988 Connectionist Models Summer School*, Morgan Kaufmann, 1988, pp. 52–59 (cit. on p. 19).

[26] I. Loshchilov and F. Hutter. "Decoupled Weight Decay Regularization." arXiv: 1711.05101 [cs.LG], Accessed: Jun. 20, 2026. [Online]. Available: http://arxiv.org/abs/1711.05101 pre-published (cit. on p. 20).

[27] K. He, X. Zhang, S. Ren, and J. Sun. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." arXiv: 1502.01852 [cs.CV], Accessed: Jul. 15, 2026. [Online]. Available: http://arxiv.org/abs/1502.01852 pre-published (cit. on p. 20).

[28] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, JMLR Workshop and Conference Proceedings, Mar. 31, 2010,

pp. 249–256. Accessed: Jul. 15, 2026. [Online]. Available: https://proceedings.mlr.press/v9/glorot10a.html (cit. on p. 20).

[29] D. C. Hoaglin, F. Mosteller, and J. W. Tukey, Eds., *Understanding Robust and Exploratory Data Analysis* (Wiley Classics Library), Wiley classics library ed. New York: Wiley, 2000, 447 pp., ISBN: 978-0-471-38491-5 (cit. on p. 21).

[30] C. Gini, *Variabilità e Mutabilità: Contributo Allo Studio Delle Distribuzioni e Delle Relazioni Statistiche. [Fasc. I.]* (Studi Economico-Giuridici Pubblicati per Cura Della Facoltà Di Giurisprudenza Della R. Università Di Cagliari). Tipogr. di P. Cuppini, 1912. [Online]. Available: https://books.google.es/books?id=fqjaBPMxB9kC (cit. on p. 26).

[31] B. Hanin and D. Rolnick. "Complexity of Linear Regions in Deep Networks." arXiv: 1901.09021 [stat.ML], Accessed: Jun. 7, 2026. [Online]. Available: http://arxiv.org/abs/1901.09021 pre-published (cit. on p. 31).

[32] T. Tanay and L. Griffin. "A Boundary Tilting Persepective on the Phenomenon of Adversarial Examples." arXiv: 1608.07690 [cs.LG], Accessed: Jul. 1, 2026. [Online]. Available: http://arxiv.org/abs/1608.07690 pre-published (cit. on p. 48).

[33] A. I. Humayun, R. Balestriero, and R. Baraniuk. "Deep Networks Always Grok and Here is Why." arXiv: 2402.15555 [cs.LG], Accessed: Jun. 28, 2026. [Online]. Available: http://arxiv.org/abs/2402.15555 pre-published (cit. on p. 49).

[34] M. Raghu, B. Poole, J. Kleinberg, S. Ganguli, and J. Sohl-Dickstein. "On the Expressive Power of Deep Neural Networks." arXiv: 1606.05336 [stat], Accessed: Mar. 24, 2026. [Online]. Available: http://arxiv.org/abs/1606.05336 pre-published (cit. on p. 50).

[35] V. Bosca, T. Rask, S. Tanweer, A. R. Tawfeek, and B. Stone. "Topological Signatures of ReLU Neural Network Activation Patterns." arXiv: 2510.12700 [cs], Accessed: Mar. 24, 2026. [Online]. Available: http://arxiv.org/abs/2510.12700 pre-published (cit. on p. 52).

[36] G.-H. Lee, D. Alvarez-Melis, and T. S. Jaakkola. "Towards Robust, Locally Linear Deep Networks." arXiv: 1907.03207 [cs.LG], Accessed: Jun. 17, 2026. [Online]. Available: http://arxiv.org/abs/1907.03207 pre-published (cit. on p. 53).

[37] D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar, and N. Srebro. "The Implicit Bias of Gradient Descent on Separable Data." arXiv: 1710.10345 [stat.ML], Accessed: Jul. 6, 2026. [Online]. Available: http://arxiv.org/abs/1710.10345 pre-published (cit. on p. 54).

Declaration on the Use of Artificial Intelligence Tools

During the preparation of this thesis, I used the generative artificial intelligence tools ChatGPT by OpenAI and Claude by Anthropic for linguistic support. Their use was limited to improving wording, sentence structure, clarity, and readability, as well as suggesting alternative formulations.

The research questions, methodology, mathematical derivations, analyses, interpretations, and conclusions presented in this thesis are my own. I critically reviewed and revised all AI-assisted suggestions and take full responsibility for the accuracy, originality, and final content of this thesis. AI-generated responses were not treated as academic sources.

Michael Kittel, Münster, July 16, 2026

Declaration of Academic Integrity

I hereby confirm that this thesis, entitled

Geometric Differences in Activation Regions
of MLPs under Different Loss Functions

is solely my own work and that I have used no sources or aids other than the ones stated. All passages in my thesis for which other sources, including electronic media and AI Tools, have been used, be it direct quotes or content references, have been acknowledged as such and the sources cited. I am aware that plagiarism is considered an act of deception which can result in sanction in accordance with the examination regulations.

Michael Kittel, Münster, July 16, 2026

I consent to having my thesis cross-checked with other texts to identify possible similarities and to having it stored in a database for this purpose. I confirm that I have not submitted the following thesis in part or whole as an examination paper before.

Michael Kittel, Münster, July 16, 2026